KUBERNETES NETWORKING



learnk8s

Table of contents

1. Introduction **5**

2. Exploring the cluster network **8**

Cluster network 9

The bridge 17

How Flannel works 22

Tracing the response 26

Summary 29

Tidy up 30

3. Endpoints **31**

Inspecting endpoints 32

Endpoints objects and IP addresses:ports pairs 34

Custom Endpoint 39

External endpoints 42

Summary 44

Tidy up 45

4. Kube-proxy **46**

The ClusterIP service 47
Intercept and rewrite 49
Intercept and rewrite at scale 54
Summary 55
Tidy up 56

5. Service discovery 57
Exploring CoreDNS 58
CoreDNS and DNS query of an existing name 63
DNS and namespaces 66
Headless service 68
Summary 74
Tidy up 74

6. Network policies 75
LIMIT traffic to an application 76
DENY all non-whitelisted traffic to a namespace 85
DENY external egress traffic 88
Summary 90

Tidy up 91

7. Ingress controller 92

Ingress without an Ingress (controller) 94

Multiple Ingress controllers 98

Anatomy of an ingress manifest 101

Ingress controller in detail 104

Summary 111

Tidy up 112

8. Service meshes 113

Creating a cluster and installing Istio 114

Consuming the API 119

Inspecting the dependency graph with Kiali 121

Canary release 126

Summary 132

Chapter 1

Introduction

In this module, you will explore the Kubernetes networking model.

Don't let the size of this module scare you: networking is a vast and rich subject, but there are a few ways to tackle it.

If your interest is primarily in application development, we recommend focusing on the following chapters:

- Service discovery
- Network policies
- Ingress
- Service mesh

If you want to immerse yourself in Kubernetes and want to tackle all chapters, you can organise your study as follow:

- First, cover the chapter on internal cluster networking (from Flannel to endpoints).
- Then, tackle Services (from kube-proxy to service discovery).
- And finally, tackle external networking (Ingress and Istio).

Before starting, please be aware that you must SSH into several nodes.

To keep track of what command is sent to a specific node, current host or container, the code snippets follow this convention:

When the command is local, the title of the windows is

`bash` or `PowerShell`:

bash

```
$ local command
```

PowerShell

```
PS> local command
```

When the command is executed against a remote instance, the code snippet is titled `bash@<remote location>`.

bash@node1

```
$ remote command on node1
```

Let's get started!

And remember, if you are stuck or have questions, feel free to ping us on Slack.

Chapter 2

Exploring the cluster network

In this chapter, you will explore the cluster network built using a software-defined network (Flannel).

Let's start minikube with two nodes:

bash

```
$ minikube start --nodes=2 --cni=flannel --driver=docker \
  --kubernetes-version=v1.26.2 \
  --container-runtime=containerd
✨  Automatically selected the docker driver
👍  Starting control plane node minikube in cluster minikube
🚜  Pulling base image ...
🔥  Creating docker container (CPUs=2, Memory=1988MB) ...

👍  Starting worker node minikube-m02 in cluster minikube
🚜  Pulling base image ...
🔥  Creating docker container (CPUs=2, Memory=1988MB) ...
🔎  Verifying Kubernetes components...
🏄  Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
```

PowerShell

```
PS> minikube start --nodes=2 --cni=flannel --driver=docker `
  --kubernetes-version=v1.26.2 `
  --container-runtime=containerd
✨  Automatically selected the docker driver
👍  Starting control plane node minikube in cluster minikube
🚜  Pulling base image ...
🔥  Creating docker container (CPUs=2, Memory=1988MB) ...

👍  Starting worker node minikube-m02 in cluster minikube
🚜  Pulling base image ...
🔥  Creating docker container (CPUs=2, Memory=1988MB) ...
🔎  Verifying Kubernetes components...
🏄  Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
```

Cluster network

In Kubernetes, you can find at least two different networks:

1. **The network that connects all the nodes in the cluster.**

2. **The network that connects all Pods.**

The two networks can also be combined: Pods and Nodes could share the same IP address ranges.
That's because Kubernetes doesn't prescribe a way to configure your networking stack.
However, it has two strict requirements:

1. In the cluster network, all pods can reach all other pods.
2. Any agent on the node can reach any Pod on that node.

To explore the cluster network, let's create a DaemonSet with the [netshoot container.]

netshoot.yaml

```yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: netshoot
spec:
  selector:
    matchLabels:
      app: netshoot
  template:
    metadata:
      labels:
        app: netshoot
    spec:
      containers:
      - name: netshoot
        image: ghcr.io/learnk8s/netshoot:2023.03
        command: ["sleep"]
        args: ["86400"]
```

A DaemonSet is a kubernetes object, that is similar in goal to Deployment (it watches over Pods), but it makes sure that there is exactly one Pod on each Node.

You can submit it to the cluster with:

```bash
$ kubectl apply -f netshoot.yaml
daemonset.apps/netshoot created
```

Let's verify that you have two Pods *Running*, one for each node:

```bash
$ kubectl get pods -o wide
NAME              READY   STATUS    IP            NODE
netshoot-xc96c    1/1     Running   10.244.0.3    minikube
netshoot-6xh25    1/1     Running   10.244.1.2    minikube-m02
```

There are two pieces of information that you should pay attention to:

1. The IP address of the Pod (`10.244.0.3` for the first and `10.244.1.2` for the second Pod).
2. The Node that hosts the Pod.

Here is a visual representation of the current setup:

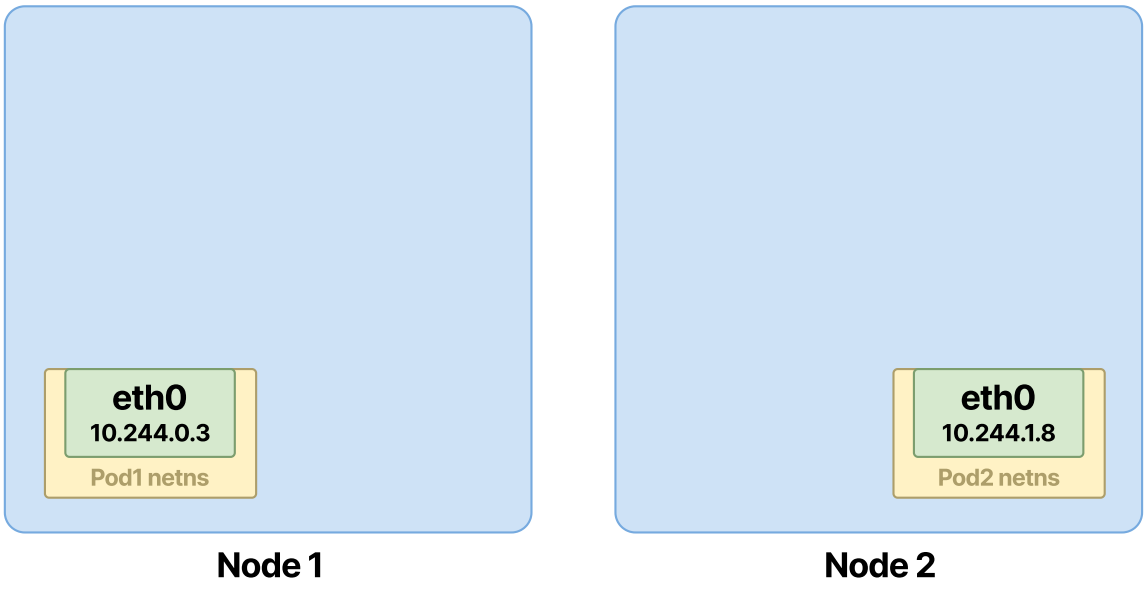


Fig. Two pods on two nodes

Let's attach to the container in the first node with:

```bash
$ kubectl exec -ti <netshoot-pod-name-node-1> -- bash
```

You can see the network interface attached to the Pod with:

```
bash@netshoot

$ ip -4 -br addr ls eth0
eth0@if6         UP             10.244.0.3/24
```

Notice how the Pod has an `eth0` ethernet device and:

- The IP address is `10.244.0.3`,
- The subnet is `10.244.0.0/24`.

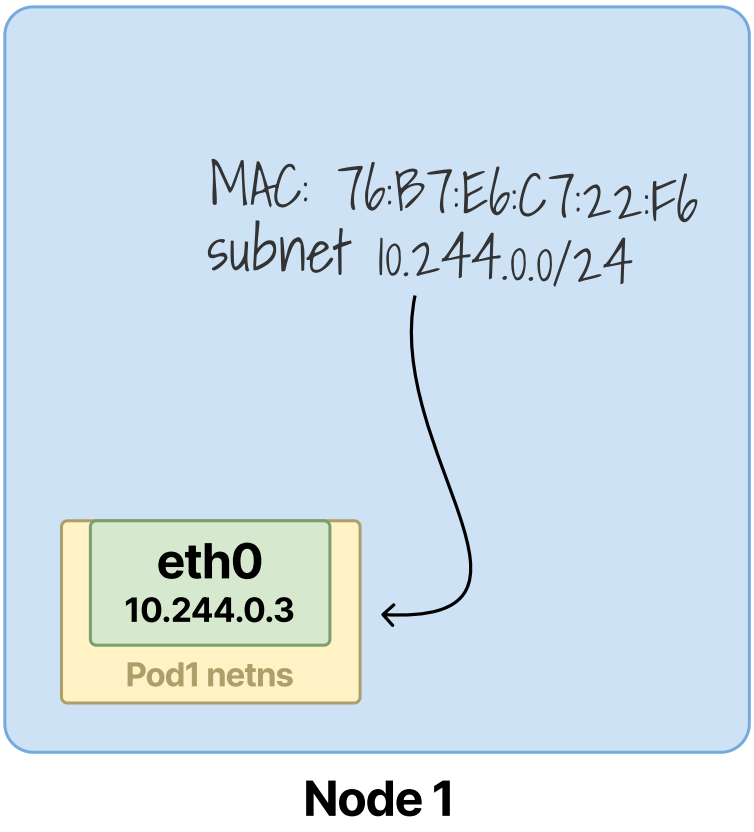


Fig. Pod's IP address, subnet and MAC address

Let's see how routing is configured inside the Pod with:

bash@netshoot

```
$ ip ro ls
default via 10.244.0.1 dev eth0
10.244.0.0/24 dev eth0 proto kernel scope link src 10.244.0.3
10.244.0.0/16 via 10.244.0.1 dev eth0
```

As the first and last line are redundant, the rules can be shortened to:

1. `10.244.0.x` : go direct,
2. everything else: go via the gateway `10.244.0.1` .

What does that mean?

If the Pod issues a request to a Pod on the same node (`10.244.0.x` subnet), the traffic will reach the pod directly.

If the Pod requests an address outside this range, the traffic is forwarded to the gateway instead.

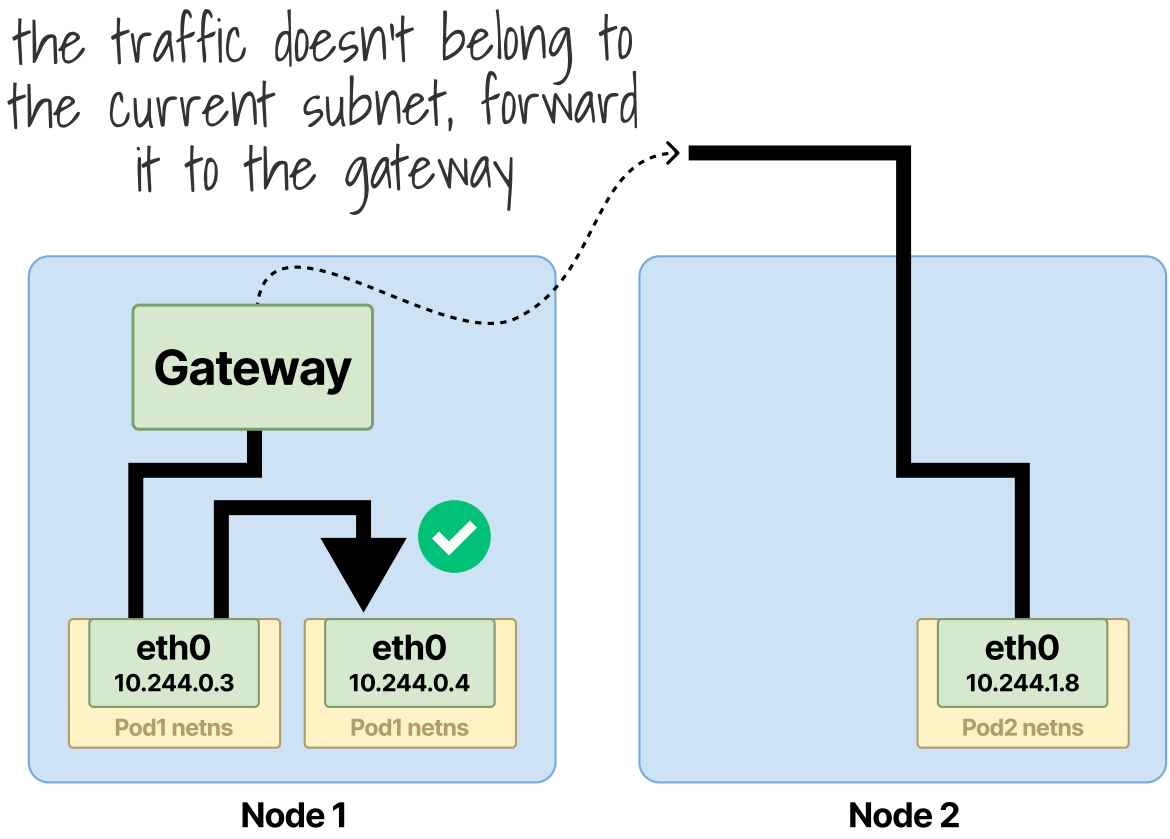


Fig. The non-local traffic is forwarded to the gateway

But who is the gateway?

Let's answer that question by opening a new terminal session and logging inside the cluster node.

bash

```
$ minikube ssh --node=minikube
```

```
bash@minikube

$ ip -4 -br addr ls
cni0             UP             10.244.0.1/24
# truncated output
```

Wait, `10.244.0.1` is the IP of the gateway!

So all traffic not local to the node is forwarded to the `cni0` interface on the node.

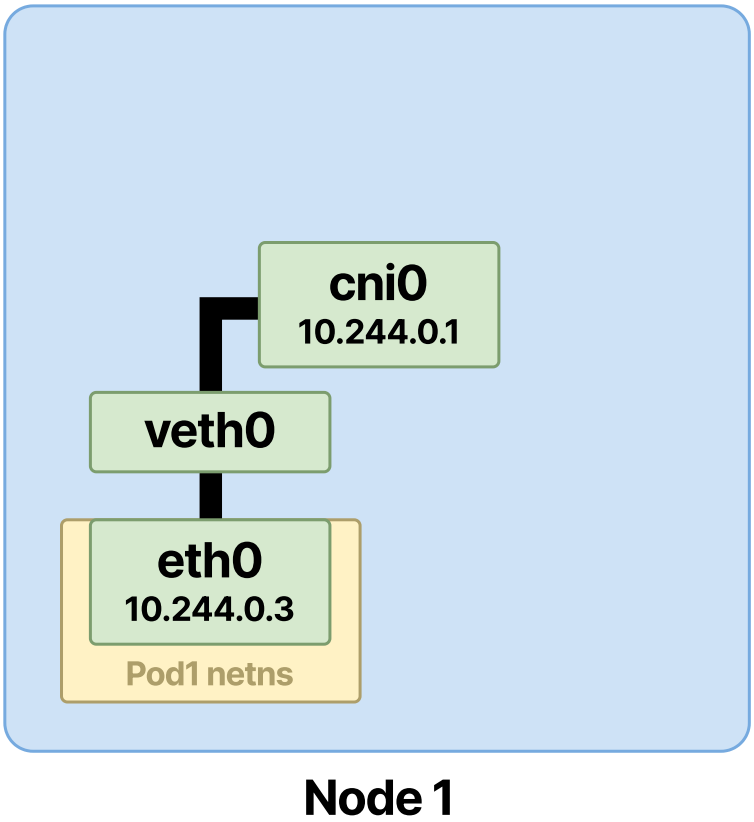


Fig. The cni0 is a bridge that connect all containers

Let's test this.

The bridge

You can listen to all the traffic passing through that interface with:

bash@minikube

```
$ sudo apt update && sudo apt install -y tcpdump
sudo tcpdump -qnt -i cni0 icmp
listening on cni0, link-type EN10MB (Ethernet), capture size 262144 bytes
```

Notice how there is no traffic flowing at the moment.

Return to the netshoot session, and let's ping the other Pod's IP address (in this case, `10.244.1.2`).

If the traffic flows through `cni0`, you should be able to see those packets flowing in TCPdump.

bash@netshoot

```
$ ping 10.244.1.2
PING 10.244.1.2 (10.244.1.2): 56 data bytes
64 bytes from 10.244.1.2: seq=0 ttl=62 time=0.614 ms
64 bytes from 10.244.1.2: seq=1 ttl=62 time=0.126 ms
# truncated output
```

Pay attention to the tcpdump session; the same packets

arrive at the `cni0` interface!

bash@minikube

```
IP 10.244.0.3 > 10.244.1.2: ICMP echo request, id 28208, seq 1, length 64
IP 10.244.1.2 > 10.244.0.3: ICMP echo reply, id 28208, seq 1, length 64
```

You just confirmed that if you ping the other Pods' IP address `10.244.1.2`, the ICMP ping packets are forwarded to `cni0`.

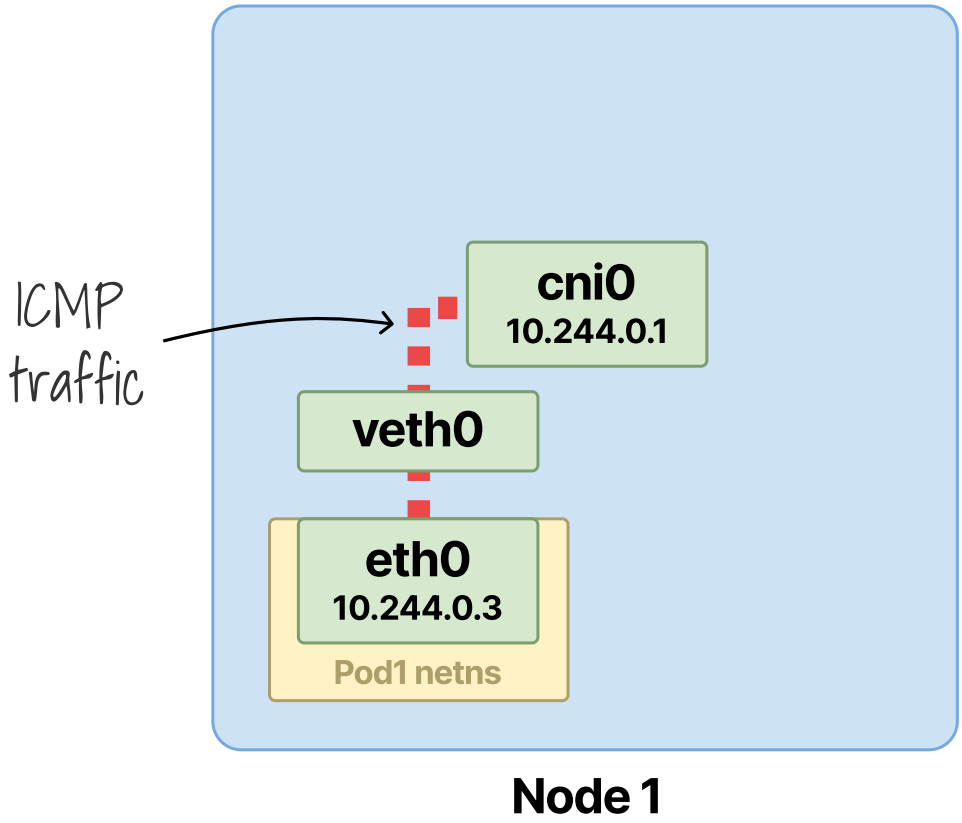


Fig. Traffic flowing from the Pod to the cni0 bridge

But `cni0` isn't a physical network device.
So how does `cni0` *connect to the address* `10.244.1.2` *?*
In the SSH session in the minikube node, stop the tcpdump session and let's list the routes with:

bash@minikube

```
$ ip ro ls
```

```
default via 192.168.49.1 dev eth0
10.244.0.0/24 dev cni0 proto kernel scope link src 10.244.0.1
10.244.1.0/24 via 10.244.1.0 dev flannel.1 onlink
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.49.0/24 dev eth0 proto kernel scope link src 192.168.49.2
```

We see, that the address `10.244.1.x` will be forwarded to `flannel.1`.
The primary function of this interface is to encapsulate the traffic and create a tunnel to the other nodes.

> In this case, `flannel.1` is the VXLAN interface that encapsulates Layer 2 frames in UDP. You can learn more about the VXLAN here. Flannel offers options other than VXLAN, and you can find them here.

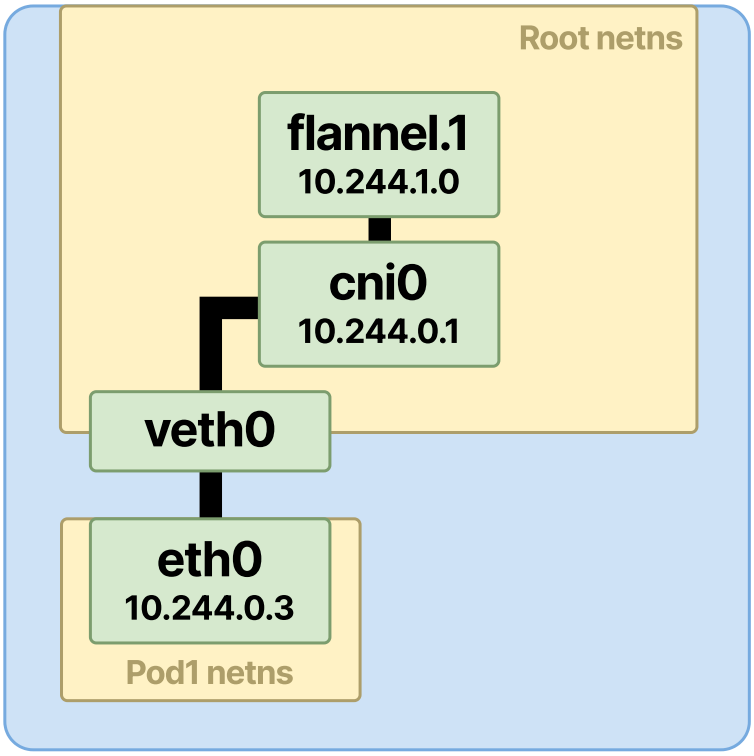


Fig. All the traffic from the cni0 bridge is forwarded to the flannel.1 interface and encapsulated

How does `flannel.1` *know which node to send the traffic to?*

It doesn't.

That's when the Flannel Pod comes into play.

How Flannel works

If you list all the Pods in the cluster, you might notice it.

bash

```
$ kubectl get pods --all-namespaces
NAMESPACE     NAME                                   READY   STATUS
default       netshoot-6xh25                         1/1     Running
default       netshoot-xc96c                         1/1     Running
kube-system   coredns-78fcd69978-jcvf2               1/1     Running
kube-system   etcd-minikube                          1/1     Running
kube-system   kube-apiserver-minikube                1/1     Running
kube-system   kube-controller-manager-minikube       1/1     Running
kube-system   kube-flannel-ds-arm64-72rll            1/1     Running
kube-system   kube-flannel-ds-arm64-zwkww            1/1     Running
kube-system   kube-proxy-4jnxh                       1/1     Running
kube-system   kube-proxy-n6sxg                       1/1     Running
kube-system   kube-scheduler-minikube                1/1     Running
kube-system   storage-provisioner                    1/1     Running
```

The Flannel Pod keeps track of all the Pods in the cluster and their IP addresses.

This binary matches the source to the destination and helps with the encapsulation.

At the end of the day, the packet is forwarded to `eth0` before jumping to the second node.

Let's observe the encapsulated traffic leaving the `eth0` interface.

In the SSH session with the minikube node, let's inspect the traffic with:

bash@minikube

```
$ sudo tcpdump -qnt -i eth0 udp
listening on cni0, link-type EN10MB (Ethernet), capture size 262144 bytes
```

Note how we are interested in UDP packets only now.

In the netshoot session, let's start pinging the other Pod with:

bash@netshoot

```
$ ping 10.244.1.2
PING 10.244.1.2 (10.244.1.2): 56 data bytes
64 bytes from 10.244.1.2: seq=0 ttl=62 time=0.614 ms
64 bytes from 10.244.1.2: seq=1 ttl=62 time=0.126 ms
# truncated output
```

Observe the traffic in the previous terminal:

bash@minikube

```
IP 192.168.49.2.41430 > 192.168.49.3.8472: UDP, length 106
IP 192.168.49.3.41430 > 192.168.49.2.8472: UDP, length 106
```

We can see, that on the physical `eth0` interface, the traffic becomes encapsulated by UDP on port 8472.

This traffic is actually not encrypted (just encapsulated), so if we remove the `-q` flag, tcpdump can show more:

bash@minikube

```
$ sudo tcpdump -nt -i eth0 udp
```

```
listening on cni0, link-type EN10MB (Ethernet), capture size 262144 bytes
IP 192.168.49.2.41430 > 192.168.49.3.8472: OTV, flags [I] (0x08), overlay 0, instance 1
IP 10.244.0.3 > 10.244.1.2: ICMP echo request, id 6247, seq 1, length 64
IP 192.168.49.3.41430 > 192.168.49.2.8472: OTV, flags [I] (0x08), overlay 0, instance 1
IP 10.244.1.2 > 10.244.0.3: ICMP echo reply, id 6247, seq 1, length 64
```

These 4 lines need a bit of thinking to decipher...

This is not 4 packets, only 2 packets, 2 lines each; so let's look only at the first 2 lines, which is the request direction:

- The packet goes from `192.168.49.2` to `192.168.49.3`, which are the first and second nodes of the cluster.
- It encapsulates another packet that goes from `10.244.0.3` to `10.244.1.2`, the two Pods' IP addresses.
- The body of the encapsulated packet is an "ICMP echo request".

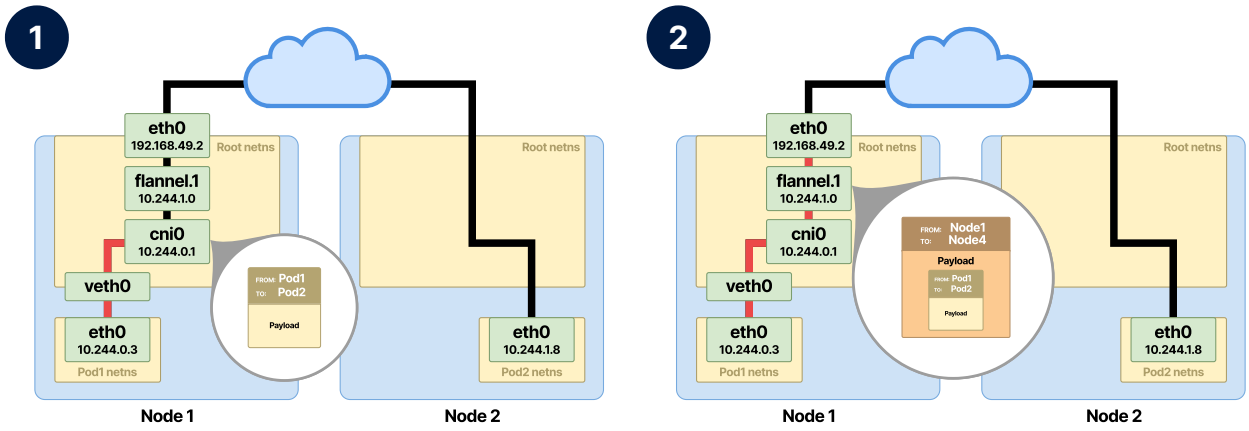


Fig. 1 A packet sent from Pod1 to Pod2.

Fig. 2 Flannel encapsulates the traffic into a UDP packet that forwards packets to nodes, not pods.

Put it all together, and you have below network devices:

NETWORK DEVICE	IP ADDRESS
eth0 (netshoot container)	10.244.0.3
cni0	10.244.0.1
flannel.1	10.244.1.0
eth0 (node)	192.168.49.2

As soon as the packet travels to the other node, it will follow the process in reverse.

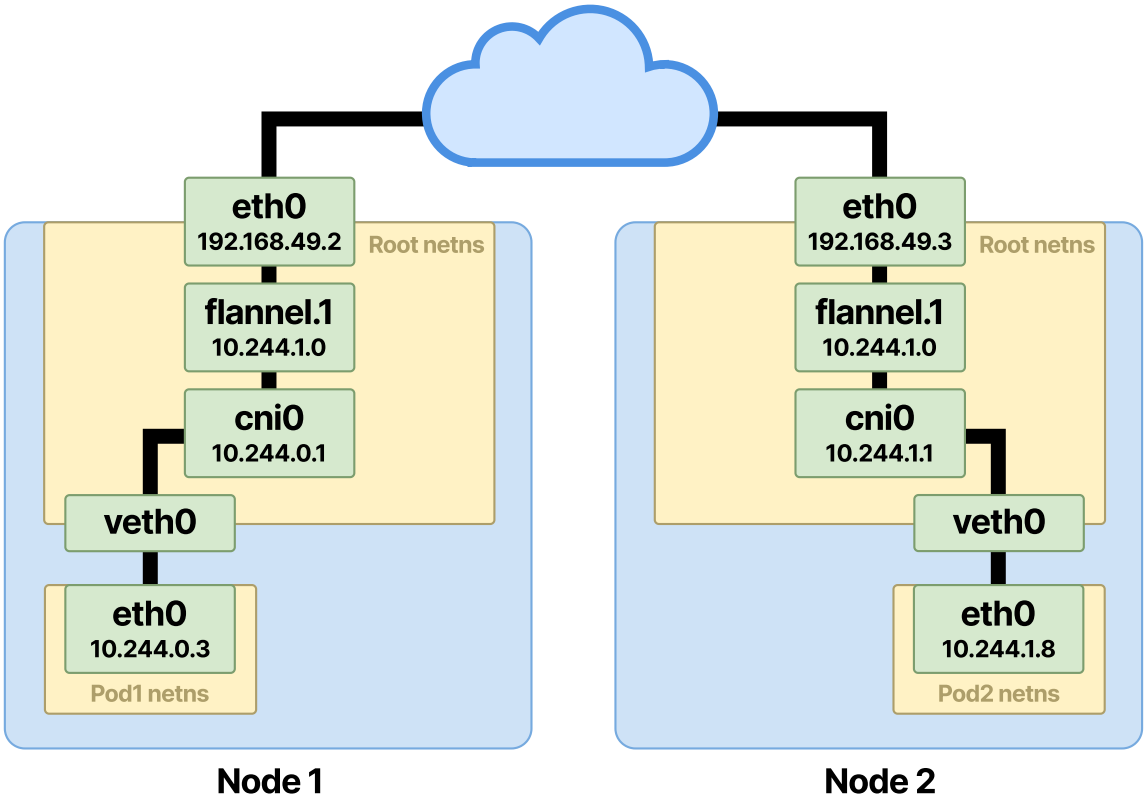


Fig. All the interfaces involved in forwarding the packet to its destination

Let's verify that it's the case.

Tracing the response

You should `exit` from the current SSH session to the first node and create another session in the second node with:

bash

```
$ minikube ssh --node=minikube-m02
```

Once you are logged in, you should install `tcpdump` and `ifconfig` with:

bash@minikube-m02

```
$ sudo apt update && sudo apt install -y tcpdump
```

You are ready to repeat the experiment from the receiving node.

What happens if you listen to UDP packets on the `eth0` *network device?*

bash@minikube-m02

```
$ sudo tcpdump -qnt -i eth0 udp
listening on cni0, link-type EN10MB (Ethernet), capture size 262144 bytes
```

There is no traffic — let's start pinging the Pod.

bash@netshoot

```
$ ping 10.244.1.2
PING 10.244.1.2 (10.244.1.2): 56 data bytes
```

```
64 bytes from 10.244.1.2: seq=0 ttl=62 time=0.614 ms
64 bytes from 10.244.1.2: seq=1 ttl=62 time=0.126 ms
# truncated output
```

You should see the following output:

bash@minikube-m02

```
IP 192.168.49.2.41430 > 192.168.49.3.8472: UDP, length 106
IP 192.168.49.3.41430 > 192.168.49.2.8472: UDP, length 106
```

Those are the same packets that you inspected on the other node!

Let's stop the tcpdump session on the `eth0` interface and focus on `cni0`.

At this point, you should expect the packet to be the original; no more UDP encapsulation.

bash@minikube-m02

```
sudo tcpdump -qnt -i cni0 icmp
listening on cni0, link-type EN10MB (Ethernet), capture size 262144 bytes
IP 10.244.0.3 > 10.244.1.2: ICMP echo request, id 25245, seq 1, length 64
IP 10.244.1.2 > 10.244.0.3: ICMP echo reply, id 25245, seq 1, length 64
```

Excellent!

You've just verified that the packets are decapsulated correctly.

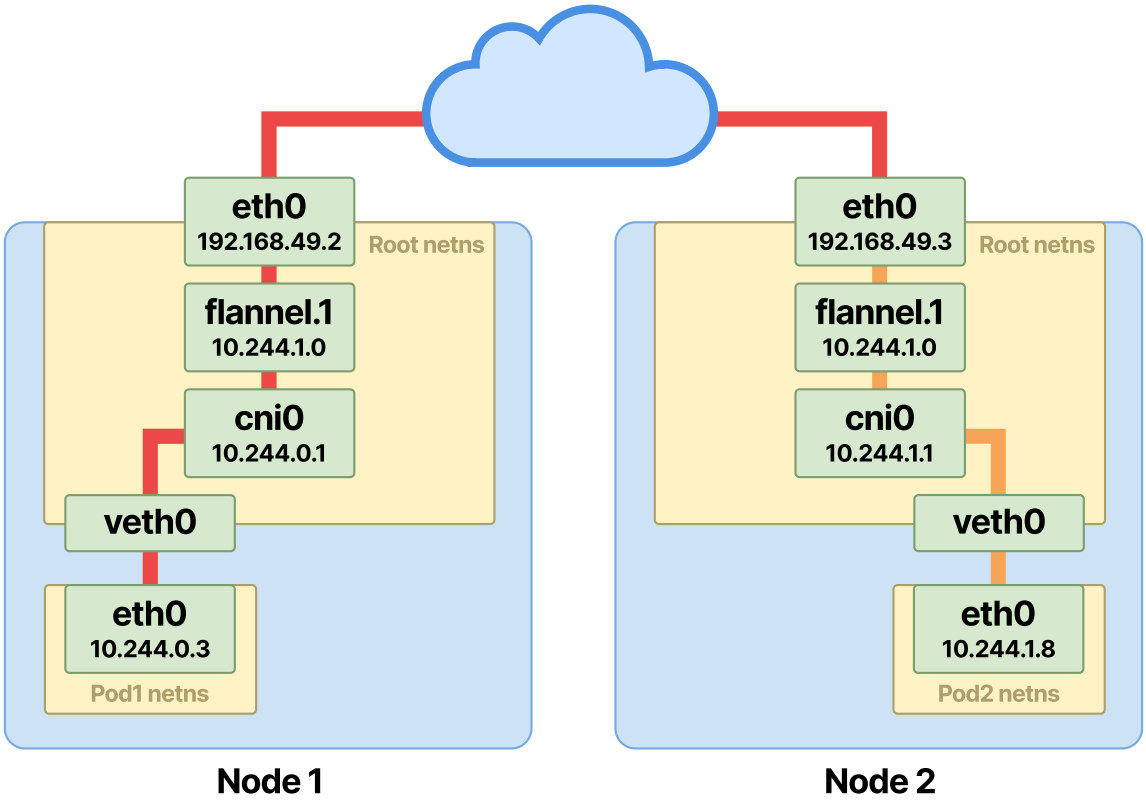


Fig. The journey of a packet forwarded to a Pod on a different node.

Summary

In this chapter, you followed the flow of a packet in the cluster travelling from one node to the other.

You also learned:

- How the containers are attached to the node.
- How Flannel encapsulates the traffic.
- How Flannel keeps the IP addresses in the cluster in sync.

Different network plugins might use other mechanisms and technologies to encapsulate the traffic.
If you are interested in a similar deep-dive, look at how Calico routes traffic in the cluster.

Tidy up

You can terminate all interactive terminal sessions and delete the netshoot DaemonSet with:

bash

```
$ kubectl delete -f netshoot.yaml
daemonset.apps "netshoot" deleted
```

Excellent, you are ready to move on to the next chapter!

Chapter 3

Endpoints

A Service in Kubernetes is an abstraction designed to route traffic to Pods.

In previous modules, you explored how the Services use a selector to match the labels on Pods.

If the selector targets `app: web`, all Pods with the label `app: web` will receive traffic (assuming their readiness probe succeeded).

But how does the Service actually work?

To answer this question, let's explore a simple deployment.

Inspecting endpoints

The following YAML file defines a Deployment for a single Nginx container.

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
spec:
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
```

```
    spec:
      containers:
      - name: nginx-container
        image: ghcr.io/learnk8s/nginx:2023.03
        ports:
          - name: web
            containerPort: 80
```

The following Service routes traffic to the Nginx Pod:

service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  selector:
    app: nginx
  ports:
    - name: http
      port: 80
      targetPort: 80
```

You can apply the manifests with:

bash

```
$ kubectl apply -f deployment.yaml,service.yaml
deployment.apps/nginx-deploy created
service/nginx-svc created
```

You can confirm that the resources are created with:

bash

```
$ kubectl get pods,services -o wide
NAME                                READY   STATUS    IP
pod/nginx-deploy-84bb6945bf-dmqtp   1/1     Running   10.244.1.9

NAME                 TYPE        CLUSTER-IP      PORT(S)   SELECTOR
service/kubernetes   ClusterIP   10.96.0.1       443/TCP   <none>
service/nginx-svc    ClusterIP   10.111.114.77   80/TCP    app=nginx
```

You can already notice a few things:

1. The service has an IP address (`10.111.114.77`) outside the Pod CIDR range. It's also not part of the CIDR for the nodes.
2. The `type` of the Service is `ClusterIP` .

Also, pay attention to the Pods' IP address: `10.244.1.9` .

Endpoints objects and IP addresses:ports pairs

Now, let's take a closer look at the Service:

bash

```
$ kubectl describe service nginx-svc
```

```
Name:              nginx-svc
Namespace:         default
Labels:            <none>
# truncated output
TargetPort:        80/TCP
Endpoints:         10.244.1.9:80
Session Affinity:  None
Events:            <none>
```

Have you noticed it?

The "Endpoints" section has the IP address and port exposed by the Nginx container.

But you didn't add the IP address when you created the Service; how was that included?

The truth is that it is not!

If you retrieve the Service from the cluster, you will notice that there's no "Endpoints" section:

bash

```
$ kubectl get svc nginx-svc -o yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
  namespace: default
spec:
  clusterIP: 10.111.114.77
  clusterIPs:
  - 10.111.114.77
  internalTrafficPolicy: Cluster
  ipFamilies:
  - IPv4
  ipFamilyPolicy: SingleStack
  ports:
  - port: 80
```

```
    protocol: TCP
    targetPort: 80
  selector:
    app: nginx
  sessionAffinity: None
  type: ClusterIP
status:
  loadBalancer: {}
```

Is the API misleading us?

No, the API is actually right, the `kubectl describe service` command just uses it in a clever way...

When you create a Service, Kubernetes automatically creates an Endpoint companion object.

bash

```
$ kubectl get services,endpoints
NAME                 TYPE        CLUSTER-IP      PORT(S)
service/kubernetes   ClusterIP   10.96.0.1       443/TCP
service/nginx-svc    ClusterIP   10.111.114.77   80/TCP

NAME                   ENDPOINTS
endpoints/kubernetes   192.168.49.2:8443
endpoints/nginx-svc    10.244.1.9:80
```

> Note how the Endpoints' name matches the Service's name.

The Endpoints contains the list of IP addresses and ports that match the Service selector.

Let's verify that the Pod's IP address and port are in that list.

```bash
$ kubectl describe endpoints nginx-svc
Name:         nginx-svc
Namespace:    default
Subsets:
  Addresses:          10.244.1.9
  Ports:
    Name     Port  Protocol
    ----     ----  --------
    <unset>  80    TCP
```

Excellent!

You've just uncovered why `kubectl describe service` can display a list of endpoints.

The command "joins" the two resources, and you can see both properties in the output.

What happens when you scale the deployment to two replicas?

```bash
$ kubectl scale deployment nginx-deploy --replicas=2
deployment.apps/nginx-deploy scaled
```

Let's recheck the endpoints:

```bash
$ kubectl get endpoints nginx-svc
NAME        ENDPOINTS
nginx-svc   10.244.1.10:80,10.244.1.9:80
```

There's one more Endpoint on the list!

What happens when you expose more than one port in the Service?

Nginx accepts traffic on port 80 and 443 for HTTPs traffic. Let's amend the Service definition to expose that port too:

service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  selector:
    app: nginx
  ports:
    - name: http
      port: 80
      targetPort: 80
    - name: https
      port: 443
      targetPort: 443
```

You can submit the resource with:

bash

```
$ kubectl apply -f service.yaml
service/nginx-svc configured
```

Can you guess how many endpoints are listed now?

bash

```
```

```
$ kubectl get endpoints nginx-svc
NAME        ENDPOINTS
nginx-svc   10.244.1.10:80,10.244.1.9:80,10.244.1.10:443 + 1 more..
```

Four!

Each Pod has two ports, and since an endpoint combines 1 IP address and one port, you end up with four endpoints.

If you expose one more Pod, the total number of endpoints will be six.

Are Endpoint objects exclusively created and managed by Kubernetes?

Let's see.

Custom Endpoint

Let's start by creating a new Service that doesn't have any `selector`:

external-service.yaml

```yaml
kind: Service
apiVersion: v1
metadata:
  name: external-svc
spec:
  ports:
    - name: web
      port: 80
```

```
      targetPort: 80
```

You can submit the resource with:

bash

```
$ kubectl apply -f external-service.yaml
service/external-svc created
```

Does Kubernetes create a companion Endpoint object for it?

bash

```
$ kubectl get endpoints
NAME         ENDPOINTS
kubernetes   192.168.49.2:8443
nginx-svc    10.244.1.10:80,10.244.1.9:80,10.244.1.10:443 + 1 more...
```

It doesn't!

What happens if you create one manually?

Let's try.

external-endpoint.yaml

```
kind: Endpoints
apiVersion: v1
metadata:
  name: external-svc
subsets:
  - addresses:
      - ip: 10.244.1.10 # replace with IP of the Nginx Pod
    ports:
```

```
    - port: 80
      name: web
```

Let's break down the resource:

- `name` has to match the name of the Service.
- `addresses` is a list of IP addresses. In this case, there's only one.
- `ports.port` and `ports.name` should also match the Service port.

You can create the Endpoint object with:

bash

```
$ kubectl apply -f external-endpoint.yaml
endpoints/external-svc created
```

Does it work?

bash

```
$ kubectl describe svc external-svc
Name:              external-svc
Namespace:         default
# truncated output
Endpoints:         10.244.1.10:80
Session Affinity:  None
Events:            <none>
```

It looks like it.

Let's issue a request to the Service from a Pod:

bash

```
$ kubectl run netshoot --rm -it --image ghcr.io/learnk8s/netshoot:2023.03 -- bash
```

And from within the netshoot container:

bash@netshoot

```
$ curl -v external-svc
*   Trying 10.102.70.197:80...
* Connected to external-svc (10.102.70.197) port 80 (#0)
# truncated output
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
# more output
```

It worked!

This time, you crafted the Endpoint yourself, though.

What if you don't use the IP address of the Pod, instead you provide an external IP?

External endpoints

Let's amend the Endpoint with:

external-endpoint.yaml

```
kind: Endpoints
apiVersion: v1
metadata:
  name: external-svc
subsets:
  - addresses:
      - ip: 40.89.244.232 # Duck Duck Go's IP address
    ports:
      - port: 80
        name: web
```

Let's apply the change with:

bash

```
$ kubectl apply -f external-endpoint.yaml
endpoints/external-svc configured
```

And let's retrieve the Endpoint from netshoot with:

bash@netshoot

```
$ curl -v external-svc
* Connected to external-svc (10.102.70.197) port 80 (#0)
> GET / HTTP/1.1
> Host: external-svc
> User-Agent: curl/7.87.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 301 Moved Permanently
< Server: nginx
< Date: Wed, 22 Mar 2023 02:07:04 GMT
< Content-Type: text/html
< Content-Length: 162
< Connection: keep-alive
< Location: https://duckduckgo.com/
# truncated output
```

It worked!

You've just found a convenient way to map resources outside of the cluster in a way that is native to Kubernetes.

All Pods can keep issuing requests to the Service, and they don't need to know that the IP doesn't belong to the cluster.

There's one last amendment that you can do to the Endpoint object.

Since the endpoints are a list, you can have more than one, and Kubernetes will load balance the requests between them.

external-endpoint.yaml

```
kind: Endpoints
apiVersion: v1
metadata:
  name: external-svc
subsets:
  - addresses:
      - ip: 40.89.244.232 # Duck Duck Go's IP address
      - ip: 46.51.197.88 # Duck Duck Go's IP address
      - ip: 176.34.135.167 # Duck Duck Go's IP address
    ports:
      - port: 80
        name: web
```

Excellent!

Summary

In this chapter, you learned:

- Services are a collection of endpoints stored in etcd.
- A collection of endpoints is not stored in the Service itself but in another object called Endpoint.
- If a Service has a selector, the Endpoint object is created automatically.
- You can craft your endpoint object and point it to any IP address — even one outside of the cluster.

In the next part, you will explore two types of Services: Headless and ClusterIP.

Tidy up

Let's exit from all the interactive sessions and remove all the resources that were created:

bash

```
$ kubectl delete deployment nginx-deploy
$ kubectl delete service nginx-svc
$ kubectl delete endpoint external-svc
$ kubectl delete service external-svc
```

Chapter 4

Kube-proxy

Kubernetes has several types of Services, but they can be divided into two groups:

1. **Services that are reachable by their IP address.**
2. **Services that don't have an IP address.**

The Headless Service is the only service with no IP address and is considered unique.
Before discussing how special it is, let's explore the others, starting with ClusterIP.

The ClusterIP service

Let's start by creating a deployment for Nginx with two replicas.

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
```

```
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx-container
        image: ghcr.io/learnk8s/nginx:2023.03
        ports:
          - name: web
            containerPort: 80
```

Let's also create a Service:

service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
```

You can apply the manifests with:

bash

```
$ kubectl apply -f deployment.yaml,service.yaml
deployment.apps/nginx-deploy created
service/nginx-svc created
```

Notice how you didn't have to specify any `type` for the Service, and Kubernetes created a Service of `type: ClusterIP` by default.

```bash
$ kubectl describe service nginx-svc
Name:              nginx-svc
Namespace:         default
Selector:          app=nginx
Type:              ClusterIP
IP:                10.107.178.17
IPs:               10.107.178.17
Port:              <unset>  80/TCP
TargetPort:        80/TCP
Endpoints:         10.244.1.16:80,10.244.1.17:80
Session Affinity:  None
```

There are a few more interesting details too:

- The Service also has two endpoints: one for each Pod.
- The Service has an IP address `10.107.178.17` that doesn't belong to any range of IP addresses in the cluster.

That's because the IP address does not exist.

Well, it exists, but it's not routable.

How does it work?

Intercept and rewrite

Kubernetes uses a clever trick to pretend that the IP address can receive traffic.

Since the traffic doesn't belong to the current subnet, it is

forwarded to the gateway.

However, as the traffic exits the node, the packet is intercepted, and the destination is rewritten.

Let's investigate this in more detail.

The intercept-and-rewrite rules use iptables, so that's an excellent place to start.

> Please note that iptables rules are the default but not the only option. Network plugins might use other strategies to implement those rules. As an example, the Cilium network plugin used eBPF.

Let's create an SSH session into the first node with:

bash

```
$ minikube ssh --node=minikube
```

Let's list the rules for the Services with:

bash@minikube

```
$ sudo iptables -L KUBE-SERVICES -t nat
Chain KUBE-SERVICES (2 references)
target                     prot destination
KUBE-SVC-NPX46M4PTMTKRN6Y  tcp  10.96.0.1      /* default/kubernetes:https cluster IP */ tcp dpt:https
KUBE-SVC-TCOU7JCQXEZGVUNU  udp  10.96.0.10     /* kube-system/kube-dns:dns cluster IP */ udp dpt:domain
KUBE-SVC-ERIFXISQEP7F7OF4  tcp  10.96.0.10     /* kube-system/kube-dns:dns-tcp cluster IP */ tcp dpt:domain
KUBE-SVC-JD5MR3NA4I4DYORP  tcp  10.96.0.10     /* kube-system/kube-dns:metrics cluster IP */ tcp dpt:9153
KUBE-SVC-HL5LMXD5JFHQZ6LN  tcp  10.107.178.17  /* default/nginx-svc cluster IP */ tcp dpt:http
KUBE-NODEPORTS             all  anywhere
```

That's the IP of the Service!

Note how the iptables rules contain a list of all the services in the cluster, even the one used internally by Kubernetes.

What's inside the `default/nginx-svc` *rule* `KUBE-SVC-HL5LMXD5JFHQZ6LN` *?*

bash@minikube

```
$ sudo iptables -L KUBE-SVC-HL5LMXD5JFHQZ6LN -t nat
Chain KUBE-SVC-HL5LMXD5JFHQZ6LN (1 references)
target                     prot source          destination
KUBE-MARK-MASQ             tcp  !10.244.0.0/16  10.102.194.46
KUBE-SEP-DUL3TOEKR4Q7XNNH  all  anywhere        anywhere      /* default/nginx-svc */ random probability 0.5
KUBE-SEP-OJQRYVIILJUTFXOB  all  anywhere        anywhere      /* default/nginx-svc */
```

If we take a closer look, you can see two additional iptables chains:

1. `KUBE-SEP-DUL3TOEKR4Q7XNNH` and
2. `KUBE-SEP-OJQRYVIILJUTFXOB` .

The comment on the first rule suggests that it is invoked with a 50% chance.

When it isn't, the chain continues executing the following statement (which is run the other 50% of the time).

You can inspect the rules with:

bash@minikube

```
$ sudo iptables -L KUBE-SEP-DUL3TOEKR4Q7XNNH -t nat
Chain KUBE-SEP-DUL3TOEKR4Q7XNNH (1 references)
target          prot source              destination
KUBE-MARK-MASQ  all  10.244.1.16         anywhere     /* default/nginx-svc */
DNAT            tcp  anywhere            anywhere     /* default/nginx-svc */ tcp to:10.244.1.16:80

$ sudo iptables -L KUBE-SEP-OJQRYVIILJUTFXOB -t nat
Chain KUBE-SEP-OJQRYVIILJUTFXOB (1 references)
target          prot source
```

```
 destination
KUBE-MARK-MASQ  all  10.244.1.17         anywhere     /* default/nginx-svc */
DNAT            tcp  anywhere           anywhere     /* default/nginx-svc */ tcp to:10.244.1.17:80
```

What are those rules?

- The `KUBE-MARK-MASQ` marks the packet to be altered later and is only relevant for external traffic. You can safely ignore this.
- The `DNAT` rule rewrites the destination as `10.244.1.17:80` and `10.244.1.16:80`.

What are those two IP addresses?

In another terminal, list the running Pods with:

bash

```
$ kubectl get pods -o wide
NAME                              READY   STATUS    IP            NODE
nginx-deploy-84bb6945bf-8lwj8     1/1     Running   10.244.1.17   minikube-m02
nginx-deploy-84bb6945bf-nptq7     1/1     Running   10.244.1.16   minikube-m02
```

Those are the IP addresses of the Pods!

Let's recap what you just discovered:

- The Service IP address is not routable.
- **As soon as the traffic tries to exit from the node, it is intercepted by the iptables rules.**
- The iptables rules are collections of chains.
- Inside the Service's chain, there is a chain for each pod that rewrites the destination to be the pod's IP address.

What happens when you scale the deployment to three replicas?

bash

```
$ kubectl scale deployment nginx-deploy --replicas=3
deployment.apps/nginx-deploy scaled
```

Let's go back to the SSH session in minikube and list the rules for the original service rule `KUBE-SVC-HL5LMXD5JFHQZ6LN`:

bash@minikube

```
$ sudo iptables -L KUBE-SVC-HL5LMXD5JFHQZ6LN -t nat
Chain KUBE-SVC-HL5LMXD5JFHQZ6LN (1 references)
target                     prot source               destination
KUBE-MARK-MASQ             tcp  !10.244.0.0/16       10.102.194.46
KUBE-SEP-DUL3TOEKR4Q7XNNH  all  anywhere             anywhere     /* default/nginx-svc */ probability 0.33
KUBE-SEP-OJQRYVIILJUTFXOB  all  anywhere             anywhere     /* default/nginx-svc */ probability 0.50
KUBE-SEP-UATTFBHL3AQ5O4P3  all  anywhere             anywhere     /* default/nginx-svc */
```

There are three entries now: one for each Pod.

Note how the probability of routing the traffic to each Pod has changed too:

- There's a 33% chance to reach the first Pod.
- Of the remaining 66% of the traffic, 50% can go to the second Pod.
- And the remaining 50% is routed to the third Pod.

If you run the maths, you will notice that every Pod has a 33% chance of being selected.

This is iptables load balancing the traffic across the Pods selected by the Service.

Intercept and rewrite at scale

As you can imagine, the more Pods you have, the more rules you need to add to keep load balancing all requests.

And remember, all rules are written in all nodes!

Let's connect to the second node in the cluster and verify that:

bash

```
$ minikube ssh --node=minikube-m02
```

You can list the iptables rules for the Services with:

bash@minikube-m02

```
$ sudo iptables -L KUBE-SERVICES -t nat
Chain KUBE-SERVICES (2 references)
target                     prot destination
KUBE-SVC-NPX46M4PTMTKRN6Y  tcp  10.96.0.1      /* default/kubernetes:https cluster IP */ tcp dpt:https
KUBE-SVC-TCOU7JCQXEZGVUNU  udp  10.96.0.10     /* kube-system/kube-dns:dns cluster IP */ udp dpt:domain
KUBE-SVC-ERIFXISQEP7F7OF4  tcp  10.96.0.10     /* kube-system/kube-dns:dns-tcp cluster IP */ tcp dpt:domain
KUBE-SVC-JD5MR3NA4I4DYORP  tcp  10.96.0.10     /* kube-system/kube-dns:metrics cluster IP */ tcp dpt:9153
KUBE-SVC-HL5LMXD5JFHQZ6LN  tcp  10.107.178.17  /* default/nginx-svc cluster IP */ tcp dpt:http
KUBE-NODEPORTS             all  anywhere
```

This is the exact same output as the other node.

Who is writing those rules, though?

Kube-proxy is in charge of keeping the rules up to date.

bash

```
$ kubectl get pods -o wide --all-namespaces
NAMESPACE     NAME                                   READY   STATUS    IP              NODE
default       nginx-deploy-84bb6945bf-8lwj8          1/1     Running   10.244.1.17     minikube-m02
default       nginx-deploy-84bb6945bf-hnv6q          1/1     Running   10.244.1.19     minikube-m02
default       nginx-deploy-84bb6945bf-nptq7          1/1     Running   10.244.1.16     minikube-m02
kube-system   coredns-78fcd69978-jcvf2               1/1     Running   10.244.0.2      minikube
kube-system   etcd-minikube                          1/1     Running   192.168.49.2    minikube
kube-system   kube-apiserver-minikube                1/1     Running   192.168.49.2    minikube
kube-system   kube-controller-manager-minikube       1/1     Running   192.168.49.2    minikube
kube-system   kube-flannel-ds-arm64-72rll            1/1     Running   192.168.49.3    minikube-m02
kube-system   kube-flannel-ds-arm64-zwkww            1/1     Running   192.168.49.2    minikube
kube-system   kube-proxy-4jnxh                       1/1     Running   192.168.49.2    minikube
kube-system   kube-proxy-n6sxg                       1/1     Running   192.168.49.3    minikube-m02
kube-system   kube-scheduler-minikube                1/1     Running   192.168.49.2    minikube
kube-system   storage-provisioner                    1/1     Running   192.168.49.2    minikube
```

Kube-proxy is deployed as a DaemonSet.
It makes sense since it has to write the rules on every cluster node.

Summary

ClusterIP is the default service in Kubernetes.

- From afar, you can pretend it's like a regular load balancer that distributes traffic to pods.
- The implementation relies on intercepting packets and routing them (randomly) to pods.
- iptables rules is the default mechanism for intercepting and rewriting in Kubernetes.

- Kube-proxy is the component in charge of writing the rules. Despite the name, there's no actual proxying of traffic.

Tidy up

Let's exit from all the interactive sessions and remove all the resources that were created:

bash

```
$ kubectl delete deployment nginx-deploy
$ kubectl delete service nginx-svc
```

Chapter 5

Service discovery

You might have already noticed, but every Service you create can be reached by its IP address and name.

We still need to discuss who is resolving the name of the Service in the cluster.

It's CoreDNS!

bash

```
$ kubectl get pods --namespace kube-system
NAME                                   READY   STATUS
coredns-78fcd69978-jcvf2               1/1     Running
etcd-minikube                          1/1     Running
kube-apiserver-minikube                1/1     Running
kube-controller-manager-minikube       1/1     Running
kube-flannel-ds-arm64-72rll            1/1     Running
kube-flannel-ds-arm64-zwkww            1/1     Running
kube-proxy-4jnxh                       1/1     Running
kube-proxy-n6sxg                       1/1     Running
kube-scheduler-minikube                1/1     Running
storage-provisioner                    1/1     Running
```

But how does the Pod know how to reach CoreDNS?

Exploring CoreDNS

DNS resolution inside a container is driven by the `/etc/resolv.conf` config file.

Let's create a container with:

```bash
$ kubectl run netshoot --rm -it --image ghcr.io/learnk8s/netshoot:2023.03 -- bash
```

And let's retrieve that file:

```bash@netshoot
$ cat /etc/resolv.conf
nameserver 10.96.0.10
search default.svc.cluster.local svc.cluster.local cluster.local
options ndots:5
```

There are three entries; let's tackle the first.

Is `10.96.0.10` *the IP address of the CoreDNS?*

```bash
$ kubectl get pods --namespace kube-system -o wide
NAME                       READY   STATUS    IP           NODE
coredns-78fcd69978-jcvf2   1/1     Running   10.244.0.2   minikube
# truncated output
```

No, it is not.

Perhaps Kubernetes uses the IP address of a Service instead, and the Service load balance the traffic to all available Pods.

```bash
$ kubectl get services --namespace kube-system
NAME       TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)
kube-dns   ClusterIP   10.96.0.10   <none>        53/UDP,53/TCP,9153/TCP
```

That's indeed the case!

So every DNS request has to pass through a Service before it reaches CoreDNS.

Which in turn reminds us that this traffic is subject to iptables rules.

What about the other two entries in the `resolv.conf` *file?*

resolv.conf

```
nameserver 10.96.0.10
search default.svc.cluster.local svc.cluster.local cluster.local
options ndots:5
```

To understand `search` and `ndots`, let's recap how the DNS resolution works for non-fully qualified names.

A fully qualified name is a name for which no local search will be executed, and the name will be treated as an absolute one during the resolution.

By convention, DNS software considers a name fully qualified if it ends with a full stop `.` and non-fully qualified otherwise.

For example, `google.com.` is a fully qualified name, but `google.com` is not.

When an application tries to resolve a name that:

1. is not fully qualified name (i.e. doesn't end with `.`).
2. contains under five dots (`ndots: 5`).

The system will try to resolve the name sequentially, going through all local search domains first; if none succeeds, will resolve it as an absolute name only at last.

Let's verify that.

Open a second terminal to netshoot, where we will be running tcpdump on port 53 (DNS):

bash

```
$ kubectl exec -it netshoot -- tcpdump -nt -i eth0 port 53
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
```

In the other terminal with the netshoot session, try to reach for a domain name that does not exist, such as `nginx-svc1`:

bash@netshoot

```
$ curl nginx-svc1
curl: (6) Could not resolve host: nginx-svc1
```

Observe the output in the tcpdump terminal:

bash

```
IP 10.244.1.5.46741 > 10.96.0.10.53: 11886+ A? nginx-svc1.default.svc.cluster.local. (54)
IP 10.244.1.5.46741 > 10.96.0.10.53: 12586+ AAAA? nginx-svc1.default.svc.cluster.local. (54)
IP 10.96.0.10.53 > 10.244.1.5.46741: 11886 NXDomain*- 0/1/0 (147)
IP 10.96.0.10.53 > 10.244.1.5.46741: 12586 NXDomain*- 0/1/0 (147)
IP 10.244.1.5.44167 > 10.96.0.10.53: 9978+ A? nginx-svc1.svc.cluster.local. (46)
IP 10.244.1.5.44167 > 10.96.0.10.53: 10178+ AAAA? nginx-svc1.svc.cluster.local. (46)
IP 10.96.0.10.53 > 10.244.1.5.44167: 9978 NXDomain*- 0/1/0 (139)
IP 10.96.0.10.53 > 10.244.1.5.44167: 10178 NXDomain*- 0/1/0 (139)
IP 10.244.1.5.56674 > 10.96.0.10.53: 50504+ A? nginx-svc1.cluster.local. (42)
IP 10.244.1.5.56674 > 10.96.0.10.53: 50804+ AAAA? nginx-svc1.cluster.local. (42)
IP 10.96.0.10.53 > 10.244.1.5.56674: 50504 NXDomain*- 0/1/0 (135)
IP 10.96.0.10.53 > 10.244.1.5.56674: 50804 NXDomain*- 0/1/0 (135)
IP 10.244.1.5.55496 > 10.96.0.10.53: 46794+ A? nginx-svc1. (28)
IP 10.244.1.5.55496 > 10.96.0.10.53: 46894+ AAAA? nginx-svc1. (28)
IP 10.96.0.10.53 > 10.244.1.5.55496: 46894 ServFail 0/0/0 (28)
IP 10.244.1.5.55496 > 10.96.0.10.53: 46894+ AAAA? nginx-svc1. (28)
IP 10.96.0.10.53 > 10.244.1.5.55496: 46894 ServFail* 0/0/0 (28)
```

There are a lot of DNS requests, for `A` (IPv4) and for `AAAA` (IPv6) records.

Notice that the queries have the search path appended:

search

```
[service-name].default.svc.cluster.local.
[service-name].svc.cluster.local.
[service-name].cluster.local.
[service-name].
```

(Please note that your cluster depending on your environment might have additional search paths after `cluster.local.` .)

Of course, since the name doesn't exist at all, all these queries fail (with NXDomain or ServFail error code).

CoreDNS and DNS query of an existing name

Let's deploy a sample application.

Create an `app.yaml` file with the following content:

app.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
spec:
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx-container
        image: ghcr.io/learnk8s/nginx:2023.03
        ports:
          - name: web
            containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
```

You can submit the deployment to the cluster with:

bash

```
$ kubectl apply -f app.yaml
pod/nginx-pod created
service/nginx-svc created
```

Let's try to resolve the Service's address in the cluster from the netshoot session:

bash@netshoot

```
$ curl nginx-svc
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
#truncated output
```

And check the tcpdump output:

bash

```
IP 10.244.1.5.40660 > 10.96.0.10.53: 39057+ A? nginx-svc.default.svc.cluster.local. (53)
IP 10.244.1.5.40660 > 10.96.0.10.53: 39257+ AAAA? nginx-svc.default.svc.cluster.local. (53)
IP 10.96.0.10.53 > 10.244.1.5.40660: 39257*- 0/1/0 (146)
IP 10.96.0.10.53 > 10.244.1.5.40660: 39057*- 1/0/0 A 10.101.222.206 (104)
```

The are only two DNS requests (IPv4 and IPv6), and they are sent in parallel.

Only the IPv4 query resolves to a valid IP address (`10.101.222.206`), this Kubernetes cluster is not using IPv6

yet.
An app can actually ask the DNS resolver library to use IPv4 only (e.g. curl's `-4` option):

bash@netshoot

```
$ curl -4 nginx-svc
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
#truncated output
```

And the tcpdump output:

bash

```
IP 10.244.1.5.40583 > 10.96.0.10.53: 28071+ A? nginx-svc.default.svc.cluster.local. (53)
IP 10.96.0.10.53 > 10.244.1.5.40583: 28071*- 1/0/0 A 10.101.222.206 (104)
```

There are a few takeaways from this experiment:

- If you misspell a Service name in the cluster, the number of DNS queries goes up a lot.
- There are four ways to reach a service:
  1. By its name, e.g. `nginx-svc`
  2. By its name and a namespace, e.g. `nginx-svc.default`
  3. By its name, namespace and service suffix, e.g. `nginx-svc.default.svc`
  4. By its fully qualified domain name, e.g.

- 4. `nginx-svc.default.svc.cluster.local.`
- By default the resolver libs try IPv6, but it can still be disabled.

It's worth exploring how the DNS works across namespaces.

DNS and namespaces

Let's create a test namespace with:

bash

```
kubectl create ns otherns
namespace/otherns created
```

Let's deploy the same Nginx Deployment and Service that you have in the default namespace.
Let's create the resources with:

bash

```
$ kubectl apply -f app.yaml -n otherns
deployment.apps/nginx-deploy created
service/nginx-svc created
```

It's worth noting that now you have two Services with the same name but in different namespaces:

bash

```
$ kubectl get svc -A
NAMESPACE     NAME         TYPE        CLUSTER-IP      PORT(S)
default       kubernetes   ClusterIP   10.96.0.1       443/TCP
default       nginx-svc    ClusterIP   10.107.178.17   80/TCP
kube-system   kube-dns     ClusterIP   10.96.0.10      53/UDP,53/TCP,9153/TCP
otherns       nginx-svc    ClusterIP   10.97.19.95     80/TCP
```

How does CoreDNS know which one is the right Service?

It doesn't.

The system's resolver lib (used by the app) uses the search path in the `/etc/resolv.conf` to issue the DNS queries.

And the first entry includes the namespace:

resolv.conf

```
nameserver 10.96.0.10
search default.svc.cluster.local svc.cluster.local cluster.local
options ndots:5
```

Let's retrieve the `resolv.conf` from the Pod in the "otherns" Namespace:

bash

```
$ kubectl get pods -n otherns
NAME                            READY   STATUS
nginx-deploy-84bb6945bf-8s847   1/1     Running

$ kubectl exec -ti -n otherns nginx-deploy-84bb6945bf-8s847 -- cat /etc/resolv.conf
nameserver 10.96.0.10
search otherns.svc.cluster.local svc.cluster.local cluster.local
options ndots:5
```

Mystery unraveled!

But can you reach the Nginx pod in the "otherns" Namespace from the "default" Namespace?

You can, as long you append the namespace's name as a suffix.

Use your original netshoot terminal to issue a request to the Service in the "otherns" Namespace:

bash@netshoot

```
$ curl nginx-svc.otherns
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
# truncated output
```

It worked!

You can have workloads in one namespace reaching for services in other namespaces.

Headless service

There's something to keep in mind when you use a Service.

Since the cluster has to pretend that the Service exists, quite a few requests are travelling the cluster before the correct Pod is reached.

Let's do a quick recap:

1. The app makes DNS queries to retrieve the IP address of the Service.
2. Since the IP address of the DNS server is a Service itself, the request is intercepted and rewritten by iptables rules.
3. When the app receives the response from CoreDNS, it issues the request to the Service's IP address.
4. Since this doesn't exist, the iptables rules will intercept the request and reroute it elsewhere.

That's many intercepting and rewriting rules for such an ordinary operation.

You can avoid some of that if you use a Headless service.

The Headless Service differs from all the other Services because it does not expose an IP address.

The only way to reach it is by issuing a DNS request to the CoreDNS (or retrieving the list of endpoints from the API).

Let's see what that means.

You can create an nginx Pod in the current namespace with:

bash

```
$ kubectl create deployment nginx-headless --image ghcr.io/learnk8s/nginx:2023.03
deployment.apps/nginx-headless created
```

The command already applied the `run: nginx-headless` label to the pod:

bash

```
$ kubectl get pods --show-labels
NAME                              READY   STATUS    LABELS
nginx-deploy-67b6b66d76-fzxhw     1/1     Running   app=nginx
nginx-headless-56cf9ddf88-s6d4g   1/1     Running   app=nginx-headless
# truncated output
```

You can create a Headless service that targets the Pod with:

service-headless.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-headless
spec:
  clusterIP: None
  selector:
    app: nginx-headless
  ports:
    - port: 80
      targetPort: 80
```

Submit the resource to the cluster with:

bash

```
$ kubectl apply -f service-headless.yaml
service/nginx-headless created
```

Let's confirm that the service has no IP address assigned:

bash

```
$ kubectl get services
```

```
NAME              TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)
kubernetes        ClusterIP   10.96.0.1        <none>        443/TCP
nginx-headless    ClusterIP   None             <none>        80/TCP
nginx-svc         ClusterIP   10.102.220.242   <none>        80/TCP
```

Let's check that the Service still works as usual.

Start a netshoot container (or use the existing one if it's still running):

bash

```
$ kubectl run netshoot --rm -it --image ghcr.io/learnk8s/netshoot:2023.03 -- bash
```

And retrieve the Service with:

bash@netshoot

```
$ curl -v nginx-headless
*   Trying 10.244.0.4:80...
* Connected to nginx-headless (10.244.0.4) port 80 (#0)
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
# truncated output
```

The output contains something rather interesting!

The Service has no IP address, but the request is routed to `10.244.0.4:80`.

What is that IP address?

bash

```
$ kubectl get pods -o wide
NAME                              READY   STATUS    IP            NODE
nginx-headless-56cf9ddf88-s6d4g 1/1     Running   10.244.0.4    minikube
```

It's the Pod's IP address.

It turns out that for every DNS query, the Headless Service replies directly with the IP address of the Pod.

In the netshoot session, let's check that it's the case:

bash@netshoot

```
$ nslookup nginx-headless
Server:    10.96.0.10
Address:   10.96.0.10#53

Name:      nginx-headless.default.svc.cluster.local
Address:   10.244.0.4
```

That's indeed the case.

What happens when you scale the number of Pods?

bash

```
$ kubectl scale deployment nginx-headless --replicas 3
deployment.apps/nginx-headless scaled
```

If you go back to the busybox session and issue the same query to the DNS, you should see the following output:

bash@netshoot

```
$ nslookup nginx-headless
Server:    10.96.0.10
Address:   10.96.0.10#53

Name:      nginx-headless.default.svc.cluster.local
Address:   10.244.0.4
Name:      nginx-headless.default.svc.cluster.local
Address:   10.244.0.5
Name:      nginx-headless.default.svc.cluster.local
Address:   10.244.1.25
```

The three IP addresses of the Pods are included in the answer!

There are a few advantages when CoreDNS replies with all the IP addresses:

- You can pick one IP address and route the traffic to it.
- There isn't a second intercept and rewrite step since you have the IPs already.
- CoreDNS load balances requests by shuffling the list of IP addresses.

But there are also a few disadvantages:

- If your application caches the response from the DNS, it might have a stale list of IP addresses.
- If you don't respect TTLs, you might end up in a similar scenario where the IP address isn't valid anymore.

If none of the above applies, you will find the Headless Service a better option for your cluster.

Summary

In this chapter, you learned:

- How CoreDNS handles all DNS queries in the cluster.
- How DNS queries are constructed.
- What is a Headless service, and how to use it.
- What happens when you issue a DNS query for a ClusterIP and Headless service.

Tidy up

Exit from all interactive sessions.

Deleting deployments and services is unnecessary since you will create a new cluster in the next chapter.

Chapter 6

Network policies

For this part of the course, you must create a new cluster with a network plugin supporting network policies.

> Unfortunately, Flannel does not support Network Policies.

You can delete the cluster with:

bash

```
$ minikube delete
💀  Removed all traces of the "minikube" cluster.
```

Let's start Minikube with another network plugin: Calico.

bash

```
$ minikube start --cni=calico --driver=docker
😄  minikube v1.24.0 on Darwin 11.6 (arm64)
✨  Automatically selected the docker driver
👍  Starting control plane node minikube in cluster minikube
🚜  Pulling base image ...
🔥  Creating docker container (CPUs=2, Memory=1988MB) ...
🏄  Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
```

LIMIT traffic to an application

Network policies are convenient to restrict the traffic allowed to reach specific Pods.

For example, you should limit what microservice has access to the backend API.

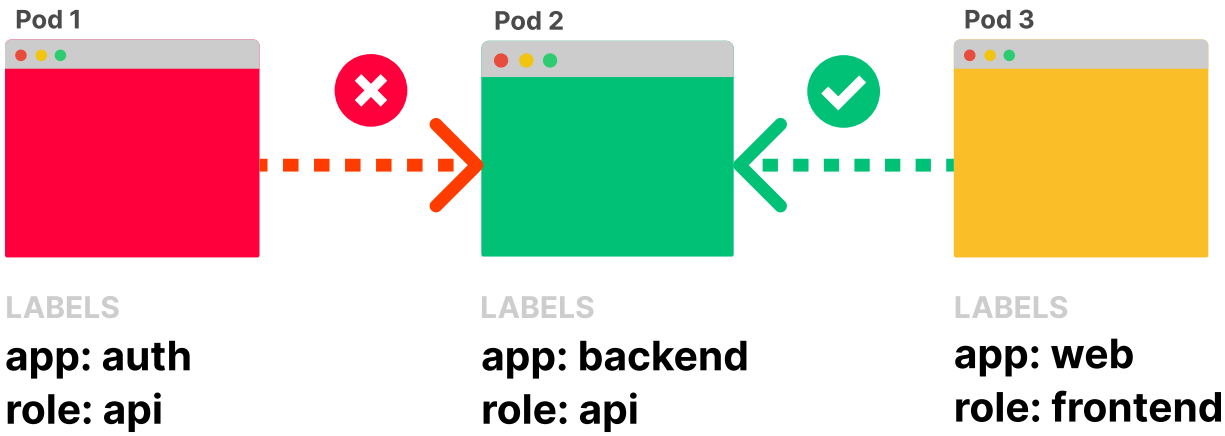


Fig. LIMIT traffic

Let's pretend your application is a REST API server, marked with labels `app: backend` and `role: api`:

bash

```
$ kubectl run backend --image=ghcr.io/learnk8s/nginx:2023.03 \
```

```
  --labels app=backend,role=api --expose --port 80
service/backend created
pod/backend created
```

```
bash                                                    — □ X

PS> kubectl run backend --image=ghcr.io/learnk8s/nginx:2023.03 `
  --labels app=backend,role=api --expose --port 80
service/backend created
pod/backend created
```

If you want to restrict the incoming traffic and allow only frontend Pods with the label `app: web` to be able to reach the backend Pod, you can have the following policy:

api-allow.yaml

```yaml
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata:
  name: api-allow
spec:
  podSelector:
    matchLabels:
      app: backend
      role: api
  ingress:
  - from:
      - podSelector:
          matchLabels:
            app: web
```

There are a few crucial details in this definition:

- It's an ingress rule, so only the incoming traffic is affected (all outgoing connections from the backend Pod will still

- be allowed).
- The targets of this policy are selected with a `podSelector`.
- The targets are chosen by their labels.

You can submit the definition to the cluster with:

bash

```
$ kubectl apply -f api-allow.yaml
networkpolicy.networking.k8s.io/api-allow created
```

Let's test that the Network Policy is **blocking** the traffic by running a Pod without the `app: web` label:

bash

```
$ kubectl run netshoot --rm -it --image ghcr.io/learnk8s/netshoot:2023.03 -- bash
```

Issue a request with:

bash@netshoot

```
$ curl -v -m 2 backend
*   Trying 10.98.226.58:80...
curl: (28) Connection timed out after 2001 milliseconds
```

> The `-m` flag sets the maximum waiting time for the request.

The traffic is blocked!

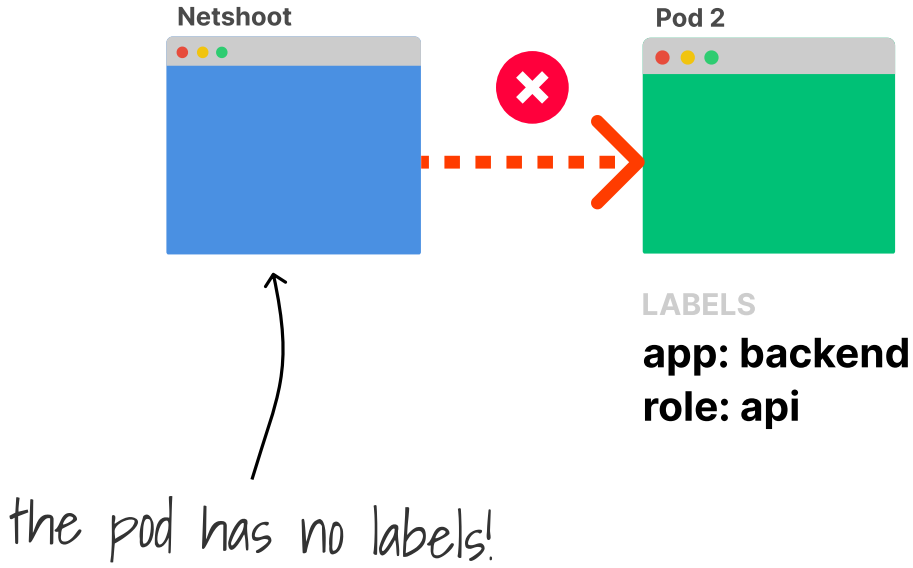


Fig. The NetworkPolicy blocks the traffic

Let's `exit` the session and try the same experiment but this time from a Pod with the `app: web` label.

bash

```
$ kubectl run netshoot --rm -it --image ghcr.io/learnk8s/netshoot:2023.03 \
  --labels app=web -- bash
```

```
bash                                                    — □ X

PS> kubectl run netshoot --rm -it --image ghcr.io/learnk8s/netshoot:2023.03
  --labels app=web -- bash
```

Issue a request with:

```
bash@netshoot

$ curl -m 2 backend
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
# truncated output
```

This time, the traffic is allowed!

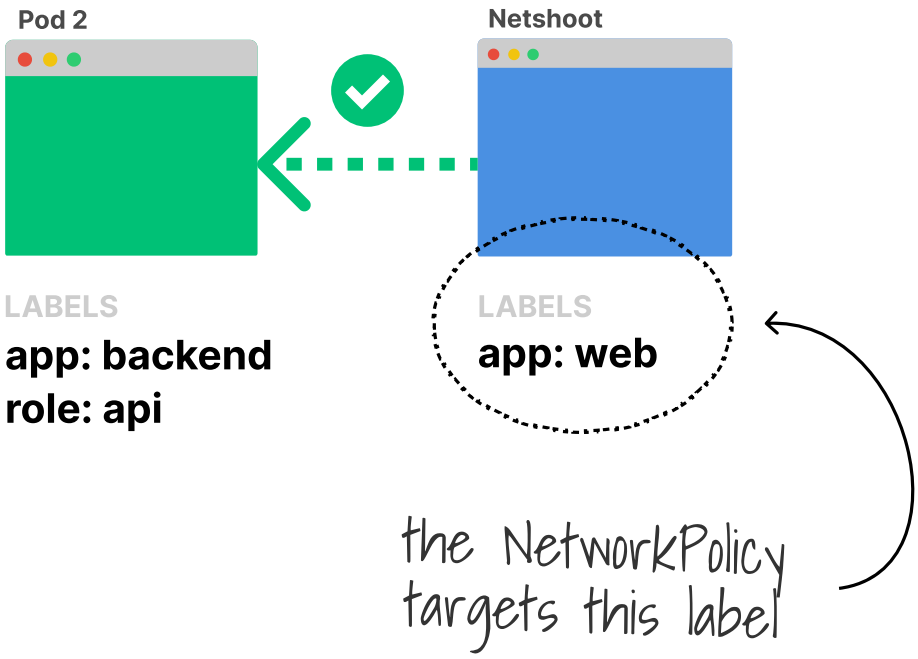


Fig. The NetworkPolicy allows the traffic

Now let's create another pod with labels `app: database` and `role: internal`:

bash

```
$ kubectl run database --image=ghcr.io/learnk8s/nginx:2023.03 \
  --labels app=database,role=internal --expose --port 80
service/database created
pod/database created
```

```
bash                                                    — □ X

PS> kubectl run backend --image=ghcr.io/learnk8s/nginx:2023.03 `
  --labels app=database,role=internal --expose --port 80
service/database created
pod/database created
```

Is the traffic allowed to reach this pod?

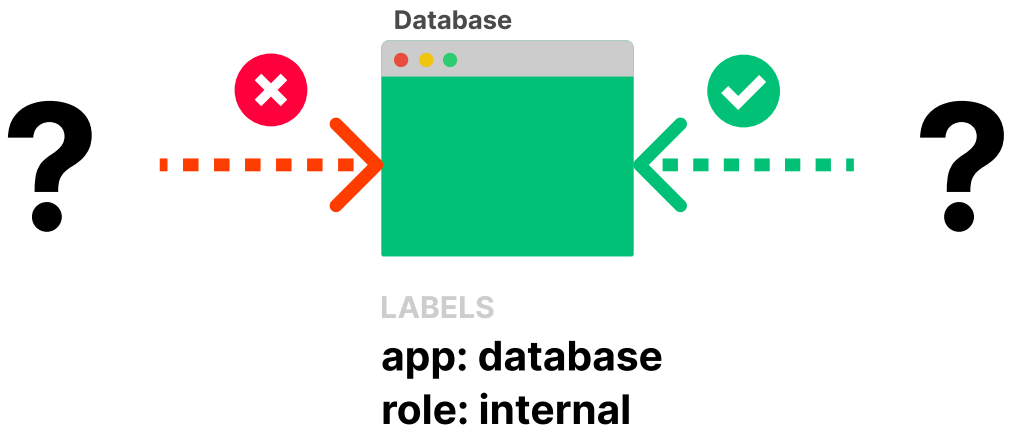


Fig. Is this pod affected by the NetworkPolicy?

Since the current network policies don't target the pod, **you**

can send all the traffic you want — with or without `app: web` label.

You can test this using the previous netshoot session or a new one:

```bash
$ kubectl run netshoot --rm -it --image ghcr.io/learnk8s/netshoot:2023.03 -- bash
```

And issuing a request with:

```bash@netshoot
$ curl -m 2 database
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
# truncated output
```

The crucial takeaway is that if there isn't any NetworkPolicy that targets the pod with an ingress rule, all traffic is allowed to that pod.

Often, this is not something you want, so let's explore ways to overcome this challenge.

Before exploring the following network policy, let's delete the current one.

```bash
```

```
$ kubectl delete netpol api-allow
networkpolicy.networking.k8s.io "api-allow" deleted
```

DENY all non-whitelisted traffic to a namespace

When working in a multitenant environment, segregating the network is often a sensible idea.

You can use the following policy to "deny all" communications in the cluster.

How can this be a good idea?

Network policies are additive — you start with a stringent rule and progressively relax the restrictions.

Let's test it.

deny-all.yaml

```yaml
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata:
  name: deny-all
spec:
  podSelector: {}
  ingress: []
```

Note a few things about this manifest:

- Since the `podSelector` is empty, it matches all Pods in the current namespace.
- There are no `ingress` rules specified. All incoming traffic will be denied to the matched pods (i.e. all).

You can apply the resource with:

bash

```
$ kubectl apply -f deny-all.yaml
networkpolicy.networking.k8s.io/deny-all created
```

Let's test it with the previous labels:

bash

```
$ kubectl run netshoot --rm -it --image ghcr.io/learnk8s/netshoot:2023.03 \
  --labels app=web -- bash
```

bash — □ X

```
PS> kubectl run netshoot --rm -it --image ghcr.io/learnk8s/netshoot:2023.03
  --labels app=web -- bash
```

Issue a request with:

bash@netshoot

```
$ curl -m 2 backend
curl: (28) Connection timed out after 2001 milliseconds

$ curl -m 2 database
curl: (28) Connection timed out after 2000 milliseconds
```

The traffic is blocked.

(With a `curl -v` test you can verify, that DNS resolution still works, as the CoreDNS pods are in the `kube-system` namespace.)

In another terminal, let's apply the previous Network Policy:

bash

```
$ kubectl apply -f api-allow.yaml
networkpolicy.networking.k8s.io/api-allow created
```

And in the netshoot session, let's repeat the experiment:

bash@netshoot

```
$ curl -m 2 backend
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
# truncated output

$ curl -m 2 database
curl: (28) Connection timed out after 2000 milliseconds
```

It worked!

You can keep relaxing rules as needed, but the network is secure by default (you can't issue any requests!).

But isn't this experiment precisely the same as the previous one?

There's a critical distinction:

- In the first example, the restrictions affected only the pod targeted by the NetworkPolicy.
- In this example, not a single pod can accept any traffic but the backend pod.

Please note that with the current "deny all" configuration, Pods can still connect to external services.

You can test this in the netshoot session:

bash@netshoot

```
$ curl -v google.com
*   Trying 74.125.24.113:80...
* Connected to google.com (74.125.24.113) port 80 (#0)
# truncated output
```

This is because all the defined rules were ingress rules, how can you block outgoing (egress) traffic?

DENY external egress traffic

A popular use case for Network policies is preventing applications from establishing connections to external networks.

Save this policy as `deny-external-egress.yaml` :

deny-external-egress.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-external-egress
spec:
  podSelector:
    matchLabels:
      app: restricted
  egress:
  - to:
    - namespaceSelector: {}
```

Let's unpack the definition:

- This policy applies to pods with an `app: restricted` label and only for outbound connections.
- Since we have an egress rule now, all outgoing traffic is blocked.
- Since the `to` field specifies an empty `namespaceSelector` **all pods in all namespaces**, all outbound traffic to other Pods in the cluster is allowed.
- And since no other rules are listed, traffic to the IP addresses outside the cluster is denied.

Submit the Network Policy to the cluster with:

bash

```
$ kubectl apply -f deny-external-egress.yaml
networkpolicy.networking.k8s.io/deny-external-egress created
```

Let's test it.

Let's create another netshoot session with the label `app: restricted`:

bash

```
$ kubectl run netshoot --rm -it --image ghcr.io/learnk8s/netshoot:2023.03 \
  --labels app=restricted -- bash
```

bash — □ X

```
PS> kubectl run netshoot --rm -it --image ghcr.io/learnk8s/netshoot:2023.03
  --labels app=restricted -- bash
```

Let's issue an external request:

bash@netshoot

```
$ curl -m 2 -v google.com
*   Trying 74.125.24.139:80...
connect to 74.125.24.139 port 80 failed: Operation timed out
# truncated output
```

The request can resolve the DNS query this time, but it cannot proceed further.

Summary

This chapter taught you how NetworkPolicies work and why you may want to use them in your cluster, particularly if you share it with other users.

- Not all Container Network Interfaces (CNIs) support NetworkPolicies.
- **You can limit traffic between the same namespace, across namespaces and for external traffic.**
- Rules apply to ingress and egress traffic.

There are two resources that we recommend exploring when it comes to network policies:

- Kubernetes Network Policy recipes is a convenient repository that recaps the most used and valuable NetworkPolicies.
- Network Policy editor is a handy editor to visualise and author Network Policies.

Tidy up

Exit from all interactive sessions.

Deleting deployments and services is unnecessary since you will create a new cluster in the next chapter.

Chapter 7

Ingress controller

You can think of the Ingress controller as a router that forwards traffic to the correct pods.

More specifically, the ingress controller is a reverse proxy that works (mainly) on the HTTP/HTTPS layer (Layer 7 of the ISO/OSI model) and lets you route traffic based on domain names, paths, etc.

Kubernetes doesn't come with one by default, so you have to install and configure an Ingress controller.

However, it provides a standardized API to define how to route the traffic in the cluster.

Let's verify that: you don't have an Ingress controller installed in the cluster.

What happens when you submit an Ingress manifest?

Let's start a new minikube cluster (you can delete the previous one with `minikube delete`):

bash

```
$ minikube start --driver=qemu --network socket_vmnet
👍  Starting control plane node minikube in cluster minikube
🐳  Preparing Kubernetes v1.25.2 ...
🔎  Verifying Kubernetes components...
🌟  Enabled addons: default-storageclass, storage-provisioner
🏄  Done! kubectl is now configured to use "minikube" by default
```

bash

```
$ minikube start --driver=kvm2
```

```
👍  Starting control plane node minikube in cluster minikube
🐳  Preparing Kubernetes v1.25.2 ...
🔎  Verifying Kubernetes components...
🌟  Enabled addons: default-storageclass, storage-provisioner
🏄  Done! kubectl is now configured to use "minikube" by default
```

```
PowerShell                                                    — □ X

PS> minikube start --driver=hyperv
👍  Starting control plane node minikube in cluster minikube
🐳  Preparing Kubernetes v1.25.2 ...
🔎  Verifying Kubernetes components...
🌟  Enabled addons: default-storageclass, storage-provisioner
🏄  Done! kubectl is now configured to use "minikube" by default
```

As soon as minikube starts, you can check that your cluster is ready with:

```
bash

$ kubectl cluster-info
Kubernetes master is running at https://192.168.64.20:8443
CoreDNS is running at <long url>
```

Ingress without an Ingress (controller)

Let's start by creating a deployment with two replicas.

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app
spec:
  replicas: 2
  selector:
    matchLabels:
      name: app
  template:
    metadata:
      labels:
        name: app
    spec:
      containers:
      - name: container1
        image: ghcr.io/learnk8s/app:1.0.0
        ports:
          - name: web
            containerPort: 8080
```

Let's also create a Service:

service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: app
spec:
  selector:
    name: app
  ports:
    - port: 80
      targetPort: 8080
```

You can apply the manifests with:

bash

```
$ kubectl apply -f deployment.yaml,service.yaml
deployment/app created
service/app created
```

Now let's create an ingress manifest file:

ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app
spec:
  rules:
    - http:
        paths:
          - backend:
              service:
                name: app
                port:
                  number: 80
            path: /
            pathType: Prefix
```

You can submit it to the cluster with:

bash

```
$ kubectl apply -f ingress.yaml
ingress.networking.k8s.io/app created
```

The cluster doesn't complain that there's no Ingress controller and lists the resource as if everything is fine:

```bash
$ kubectl get ingress
NAME   CLASS    HOSTS   ADDRESS   PORTS
app    <none>   *                 80
```

However, the request hangs if you try to access the cluster by its IP address.

```bash
$ curl $(minikube ip)
curl: (7) Failed to connect to 192.168.105.46 port 80: Connection refused
```

Let's fix this by installing the Ingress controller.

In minikube, it is as simple as enabling the Ingress addon:

```bash
$ minikube addons enable ingress
💡  ingress is an addon maintained by Kubernetes.
🔎  Verifying ingress addon...
🌟  The 'ingress' addon is enabled
```

The command installs the ingress-nginx controller based on Nginx.

You can now repeat the command and observe that the request goes through:

```bash
```

```
$ curl $(minikube ip)
<html style="background-color:#FF003C">
  <head>
    <title>v1.0.0</title>
  </head>
  <body style="display…
    <h1>v1.0.0</h1>
  </body>
</html>
```

The exact YAML definition is expected to work regardless of what Ingress controller you use.
Let's verify that.

Multiple Ingress controllers

Let's install the Traefik Ingress controller using Helm.

> You can find the instructions to install the Helm CLI here.

bash

```
$ helm repo add traefik https://traefik.github.io/charts
$ helm repo update
$ helm install traefik traefik/traefik --set ingressClass.isDefaultClass=false
Traefik Proxy v2.9.8 has been deployed successfully
on default namespace !
```

Let's visit the ingress controller with:

bash

```
$ minikube service traefik
|-----------|---------|---------------|-----------------------------|
| NAMESPACE |  NAME   |  TARGET PORT  |             URL             |
|-----------|---------|---------------|-----------------------------|
| default   | traefik | web/80        | http://192.168.105.46:32158 |
|           |         | websecure/443 | http://192.168.105.46:30761 |
|-----------|---------|---------------|-----------------------------|
```

If you visit the Traefik Ingress controller on port 80, you are greeted with the same page.

However, what if you want to differentiate which Ingress controller exposes the page?

You could uninstall the Nginx ingress controller, but it's not strictly necessary.

A cluster can have multiple Ingress controllers, and you can differentiate between them using the IngressClass.

Let's amend the ingress manifest, but this time, assign the Nginx ingressClass:

ingress-traefik.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app
spec:
  ingressClassName: nginx
  rules:
    - http:
        paths:
```

```
          - backend:
              service:
                name: app
                port:
                  number: 80
            path: /
            pathType: Prefix
```

You can submit the resource to the cluster with:

bash

```
$ kubectl apply -f ingress.yaml
ingress.networking.k8s.io/app configured
```

You can execute the previous command to visit the Traefik ingress controller with:

bash

```
$ minikube service traefik
|-----------|---------|---------------|-----------------------------|
| NAMESPACE |  NAME   |  TARGET PORT  |             URL             |
|-----------|---------|---------------|-----------------------------|
| default   | traefik | web/80        | http://192.168.105.46:32158 |
|           |         | websecure/443 | http://192.168.105.46:30761 |
|-----------|---------|---------------|-----------------------------|
```

That page shows a "404 page not found" message this time. *But what about the Nginx Ingress controller?*

bash

```
$ curl $(minikube ip)
```

```
<html style="background-color:#FF003C">
  <head>
    <title>v1.0.0</title>
  </head>
  <body style="display…
    <h1>v1.0.0</h1>
  </body>
</html>
```

It still works!

Is having multiple ingress controllers in a single cluster a good idea?

Depending on what you do, it may be.

Ingress controllers can have drastically different features.

As an example, the Kong Ingress controller could be more appropriate to serve API traffic.

The nginx-ingress controller is better suited for generic web traffic.

You can find a comparative list of all ingress controllers here.

All ingress controllers have something in common, though: the Ingress manifest.

Anatomy of an ingress manifest

Let's recap the manifest that you submitted earlier:

ingress.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app
spec:
  rules:
    - http:
        paths:
          - backend:
              service:
                name: app
                port:
                  number: 80
            path: /
            pathType: Prefix
```

This definition is packed with details, but the critical fields that you should focus on are:

1. `rules[*].http.paths[*].path` and `rules[*].http.paths[*].pathType`.
2. `rules[*].http.paths[*].backend`.

The `path` field is matched against the path of an incoming request.

The request is forwarded to the backend when there's a match.

Otherwise, it is ignored.

The `pathType` field describes how this match should be evaluated:

- When `pathType: Prefix`, the ingress controller should

- forward the traffic only if the incoming request starts with `path`. In this case, `/api` is forwarded because it includes the prefix `/`.
- When `pathType: Exact`, the incoming request is forwarded only if the path is identical to the incoming request. In this case, only `/` is forwarded.

> There's a third value, `pathType: ImplementationSpecific`, which depends on the implementation of the ingress controller.

Now let's look at the `backend` field.

The backend describes which service should receive the forwarded traffic.

In this case, the 'app' service should receive the traffic on port `80`.

But, interestingly, the traffic never uses port `80`.

This is because the controller uses endpoints to route the traffic.

Let's verify that:

bash

```
$ kubectl describe ingress app
Name:             app
Rules:
  Host        Path  Backends
  ----        ----  --------
```

```
  *
              /   app:80 (10.244.0.3:8080,10.244.0.4:8080)
Events:
  Type    Reason  From                       Message
  ----    ------  ----                       -------
  Normal  Sync    nginx-ingress-controller  Scheduled for sync
```

Those are the same endpoints listed by the Service:

bash

```
$ kubectl describe service app
Name:              app
Namespace:         default
Selector:          name=app
Type:              ClusterIP
IP Family Policy:  SingleStack
IP Families:       IPv4
IP:                10.98.155.92
IPs:               10.98.155.92
Port:              <unset>  80/TCP
TargetPort:        8080/TCP
Endpoints:         10.244.0.3:8080,10.244.0.4:8080
```

Routing directly to the pods is a great choice as the traffic skips the iptables rules defined by kube-proxy, and you can apply more advanced load-balancing strategies.

For example, you could use sticky sessions to ensure that returning visitors always land on the same pod.

Ingress controller in detail

You can think of the Ingress controller as a programmable reverse proxy.

So, who's stopping you from choosing a web server (e.g. NGINX) and deploying it to the cluster?

No one!

But that's not an ingress controller.

An Ingress controller can be called as such when:

1. Can route traffic as a reverse proxy.
2. **Integrates with the Kubernetes API** and dynamically reconfigures the proxy when there are changes to IP addresses or to the Ingress manifests.

To be more specific, the Ingress controller has to keep track of the following:

- **Ingress manifests.** This is necessary to collect the routes and routing rules.
- **Services.** Since the ingress manifest reference the service, the controller has to subscribe to changes.
- **Endpoints.** Even if the Service doesn't change, the endpoint collected by the service could.

Once the data is collected, it can reconfigure the proxy to serve the traffic.

Enough theory; let's look at an `nginx.conf` file and try to map how an Ingress controller could leverage that.

nginx.conf

```
upstream service1 {
  server 10.244.0.12:8080
}

server {
    listen                  80
    location %s {
      proxy_pass http://service1;
    }
}
```

In the first block, called `upstream`, NGINX lets you define who receives the incoming traffic.

`Upstream`s are convenient because you can group several receivers in the same block.

As an example, if you have three apps receiving the traffic, your `nginx.conf` could look like this:

nginx.conf

```
upstream service1 {
  server 10.244.0.12:8080
  server 10.244.0.13:8080
  server 10.244.0.14:8080
}

server {
    listen                  80
    location / {
      proxy_pass http://service1;
    }
}
```

`upstream`s are a grouping of IP addresses and ports, just like

the Service in Kubernetes is a collection of endpoints.

So if you were to generate the `upstream` dynamically, you could have something like this:

bash

```
$ kubectl get endpoints app -o jsonpath='{.subsets[0].addresses[*]}'
{
  "ip": "10.244.0.3",
  "nodeName": "minikube",
  "targetRef": {
    "kind": "Pod",
    "name": "app-7bf6b9cbfc-nk7w5",
    "namespace": "default",
    "uid": "2f189f94-3cb8-4f8a-90e6-6b491c06e66a"
  }
}
{
  "ip": "10.244.0.4",
  "nodeName": "minikube",
  "targetRef": {
    "kind": "Pod",
    "name": "app-7bf6b9cbfc-h68mk",
    "namespace": "default",
    "uid": "5e3b6432-cbb5-4147-9d6e-85057335f837"
  }
}
```

Wait, that's a `kubectl get endpoint` *— where is the service gone?*

You might remember that for every Service in the cluster, Kubernetes creates a companion Endpoint object.

This object contains a list of all the IP addresses and ports that belong to the service.

In this case, you need the endpoint, so there's no need to retrieve the Service.

Excellent!

The next block is `server`, and it's mostly static.

The critical part is the `location`, repeated for each incoming route and includes the `upstream` destination.

nginx.conf

```
server {
    listen                  80
    location / {
      proxy_pass http://service1;
    }
    location /api {
      proxy_pass http://service2;
    }
}
```

This sounds very similar to the Ingress manifest: a collection of rules with paths and backends.

ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app
spec:
  rules:
    - http:
        paths:
          - backend:
              service:
                name: app
                port:
                  number: 80
            path: /
            pathType: Prefix
```

You could extract the details you need with the following:

bash

```
$ kubectl get ingress app -o jsonpath='{.spec.rules[0].http.paths[0].backend}'
{
  "service": {
    "name": "app",
    "port": {
      "number": 80
    }
  }
}
```

If you could combine those commands and run them on a loop for all Ingress manifests and services, you'd have an ingress controller!

Also, the `nginx.conf` is reconfigured for every change, and the reverse proxy is restarted.

This is how the current ingress works in your cluster, so let's inspect that.

List all pods with:

bash

```
$ kubectl get pods --namespace=ingress-nginx
NAME                                        READY   STATUS
ingress-nginx-admission-create-s9z8g        0/1     Completed
ingress-nginx-admission-patch-thcvm         0/1     Completed
ingress-nginx-controller-77669ff58-9dl9f    1/1     Running
```

Attach to the running nginx controller with:

bash

```
$ kubectl exec -ti <pod id> --namespace=ingress-nginx -- bash
```

Locate the nginx config:

bash@ingress-nginx-controller

```
$ cd /etc/nginx/
```

You can inspect the full config with:

bash@ingress-nginx-controller

```
$ cat nginx.conf
# Configuration checksum: 4467338982100370 73

# setup custom paths that do not require root access
pid /tmp/nginx.pid;

daemon off;

worker_processes 1;
# truncated output
```

It's overwhelming because it is autogenerated.

You can skip to the relevant parts by searching for `app` (the ingress' name).

bash

```
$ cat nginx.conf | grep -B 10 -A 5 app
  location / {

    set $namespace      "default";
```

```
    set $ingress_name   "app";
    set $service_name   "app";
    set $service_port   "80";
    set $location_path  "/";
    set $global_rate_limit_exceeding n;
    port_in_redirect off;

    set $balancer_ewma_score -1;
    set $proxy_upstream_name "default-app-80";
    set $proxy_host          $proxy_upstream_name;
    set $pass_access_scheme  $scheme;
```

> Please note that the `nginx.conf` isn't exactly the same because the ingress-nginx project uses Lua to instrument the NGINX server.

Well done!

Summary

In this chapter, you learned:

- What is an Ingress, and how it works.
- How to provision multiple Ingress controllers in the same cluster using an IngressClass.
- Why you might want more than one Ingress controller in your cluster.

- How Services don't receive any traffic and why the Ingress route the packets directly to the pods.
- How the Ingress controller reconfigures the reverse proxy on the fly.

Tidy up

Exit from all interactive sessions.

Deleting deployments and services is unnecessary since you will create a new cluster in the next chapter.

Chapter 8

Service meshes

A popular procedure to introduce new functionality in an application is to roll out a new release to a few users.

This deployment type is called a Canary release and allows you to test your apps incrementally.

If everything works as expected, you can increase the traffic reaching the Canary release and gradually roll it out to all users.

Only a tiny percentage of users are affected if something goes wrong, and the canary release can be immediately removed.

In this chapter, you will explore what a service mesh is and how to use it to manage the traffic in your cluster.

Creating a cluster and installing Istio

Let's start by creating a local cluster with minikube (you can delete the previous cluster with `minikube delete`):

bash

```
$ minikube start --memory=4932mb --cpus=2 --driver=qemu --network socket_vmnet
👍  Starting control plane node minikube in cluster minikube
🐳  Preparing Kubernetes v1.25.2 ...
🔎  Verifying Kubernetes components...
🌟  Enabled addons: default-storageclass, storage-provisioner
🏄  Done! kubectl is now configured to use "minikube" by default
```

```
bash

$ minikube start --memory=4932mb --cpus=2 --driver=kvm2
👍  Starting control plane node minikube in cluster minikube
🐳  Preparing Kubernetes v1.25.2 ...
🔎  Verifying Kubernetes components...
🌟  Enabled addons: default-storageclass, storage-provisioner
🏄  Done! kubectl is now configured to use "minikube" by default
```

```
PowerShell                                                    — □ X

PS> minikube start --memory=4932mb --cpus=2 --driver=hyperv
👍  Starting control plane node minikube in cluster minikube
🐳  Preparing Kubernetes v1.25.2 ...
🔎  Verifying Kubernetes components...
🌟  Enabled addons: default-storageclass, storage-provisioner
🏄  Done! kubectl is now configured to use "minikube" by default
```

Now that the cluster is ready, it's time to install the service mesh.

While there are few options, the most known and used service mesh is Istio.

So, in this chapter, you will use that.

You have two options to install Istio:

1. You can use Helm or
2. You can download and use the istioctl command-line utility.

The following script uses Helm to install the prerequisites:

> If you don't have Helm installed, you can follow the instructions here.

bash

```
$ curl -s https://academy.learnk8s.io/istio.sh -o istio.sh
$ bash istio.sh
```

The script:

- Installs Istio, Prometheus and Kiali.
- Labels the default namespace for automatic sidecar injection.
- Creates an ingress gateway.
- Installs the Gateway API Custom Resource Definitions.

You can verify that all Pods are running with:

bash

```
$ kubectl get pods -n istio-system
NAME                                    READY   STATUS
istio-gateway-65bf997b9d-lckm8          1/1     Running
istiod-79d65bf5f4-47hh5                 1/1     Running
kiali-7ff96cd689-5b6dr                  1/1     Running
prometheus-server-55db4c8c5d-mvllj      2/2     Running
```

Let's move on and install the first application:

backend-v1.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend-v1
spec:
  selector:
    matchLabels:
      app: backend
      version: "1"
  template:
    metadata:
      labels:
        app: backend
        version: "1"
    spec:
      containers:
      - name: container1
        image: ghcr.io/learnk8s/app:1.0.0
        ports:
          - name: http
            containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: backend-svc
spec:
  selector:
    app: backend
  ports:
    - port: 80
      targetPort: 8080
```

You can submit the resource to the cluster with:

bash

```
$ kubectl apply -f backend-v1.yaml
deployment.apps/backend-v1 created
service/backend-svc created
```

Let's verify that the app is deployed correctly:

bash

```
$ kubectl get pod,svc
NAME                                READY   STATUS
pod/backend-v1-85f7ccc599-5lh77     2/2     Running

NAME                  TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)
service/backend-svc   ClusterIP   10.111.100.129   <none>        80/TCP
service/kubernetes    ClusterIP   10.96.0.1        <none>        443/TCP
```

Excellent!

A single Pod is running with two containers.

But wait, you define a Pod with a single container; where is the second one coming from?

Istio works by injecting a sidecar proxy into your Pods.

This sidecar container intercepts all incoming and outgoing traffic to your Pods.

Let's inspect the Pod's definition:

bash

```
$ kubectl get pod <pod-backend-v1-id> -o jsonpath="{range .spec.containers[*]}{.name}{' → '}{.image}{'\n'}{end}"
container1 → ghcr.io/learnk8s/app:1.0.0
istio-proxy → docker.io/istio/proxyv2:1.17.1
```

The sidecar is definitely there, now let's find the iptables rules that has been injected:

bash

```
$ minikube ssh
```

```
$ crictl ps | grep container1
$ crictl inspect <container_id_from_crictl_ps>
$ sudo nsenter -n -t <pid_from_crictl_inspect> iptables -L -v -t nat
# truncated output with lot of ISTIO iptables chains and rules
```

Yes, if we enter the networking namespace of the correct container, we can find the redirect rules.

All the data collected is used for metrics, but you can also use this proxy to decide where the traffic should go.

Consuming the API

Let's deploy a front-end app that consumes the API you just deployed.

frontend.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
spec:
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
      - name: frontend
        image: ghcr.io/learnk8s/netshoot:2023.03
        command: [ "/bin/sh", "-c", "--" ]
        args: [ "sleep 10; while true; do curl -s http://backend-svc/ | grep h1; sleep 1; done" ]
```

You can submit the resource to the cluster with:

```bash
$ kubectl apply -f frontend.yaml
deployment.apps/frontend created
```

The pod uses cURL to issue a request to the backend service every second.

You can verify that in the logs of the pod:

```bash
$ kubectl get pods
NAME                           READY   STATUS
backend-v1-85f7ccc599-5lh77    2/2     Running
frontend-6494c57cf7-qqgp7      2/2     Running

$ kubectl logs frontend-6494c57cf7-qqgp7
    <h1>v1.0.0</h1>
    <h1>v1.0.0</h1>
    <h1>v1.0.0</h1>
    <h1>v1.0.0</h1>
...
```

Not terribly exciting.

What's more interesting is that the proxies deployed alongside the two apps collect metrics as the traffic flows.

And the Service Mesh can derive a dependency graph of your architecture.

Inspecting the dependency graph with Kiali

Let's open the dashboard for Kiali, the Istio dashboard.
Expose the user interface with:

bash

```
$ kubectl port-forward -n istio-system svc/kiali 8080:20001
Forwarding from 127.0.0.1:8080 → 20001
Forwarding from [::1]:8080 → 20001
```

Open your browser on localhost:8080 and navigate to "Graph" you should see the following graph:

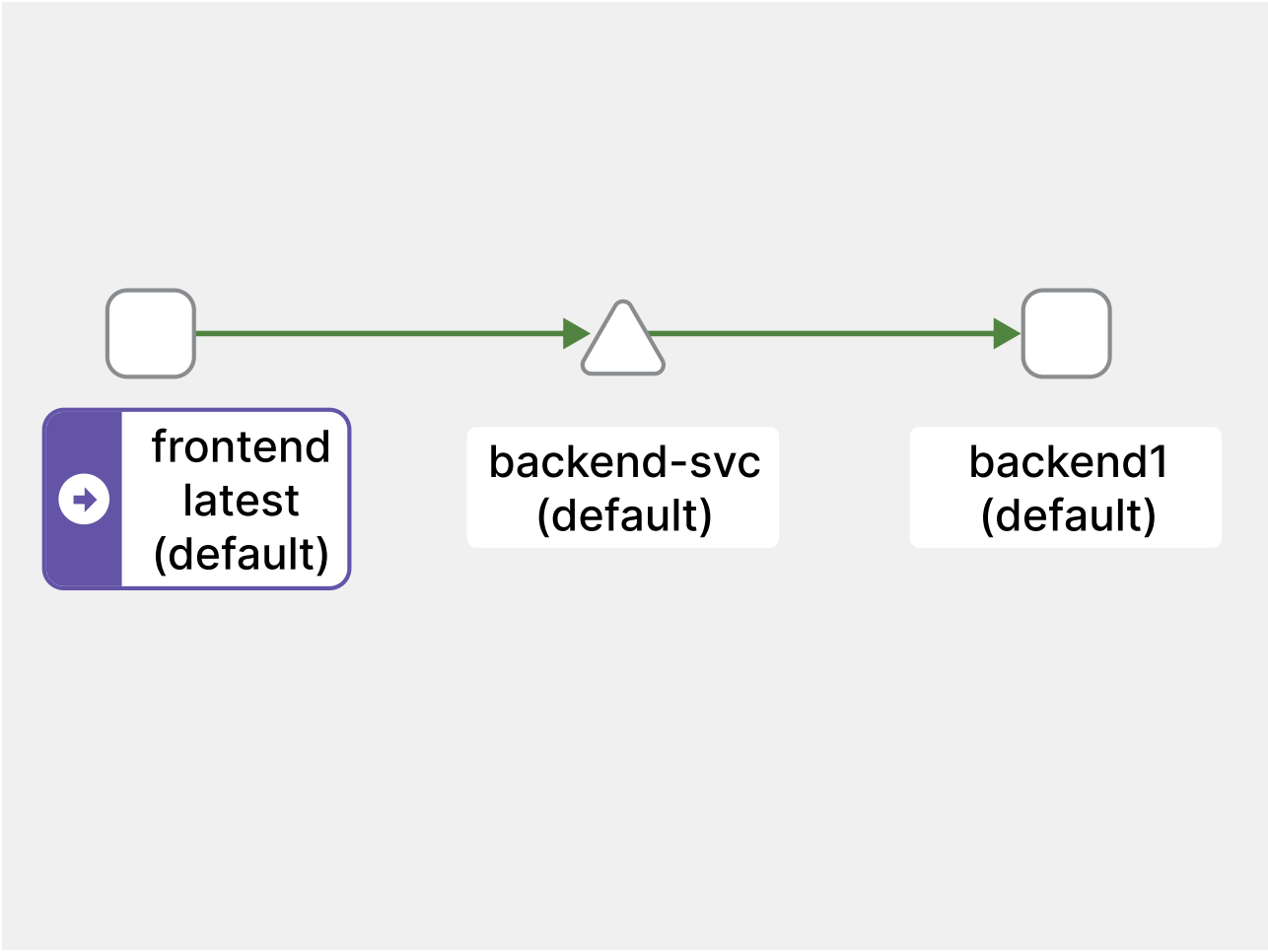


Fig. The network graph in Istio

What happens when you add more services?
Hopefully, those will be added to the dependency graph too. Let's test that by creating a second backend service:

backend-v2.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
```

```
  name: backend-v2
spec:
  replicas: 1
  selector:
    matchLabels:
      app: backend
      version: "2"
  template:
    metadata:
      labels:
        app: backend
        version: "2"
      annotations:
    spec:
      containers:
        - name: container1
          image: ghcr.io/learnk8s/app:2.0.0
          ports:
            - name: http
              containerPort: 8080
```

Before submitting the resource to the cluster, notice how the deployment shares an `app: backend` label with the previous deployment.

Since the Service selector is `app: backend`, you should expect the traffic to be distributed evenly between the two APIs.

Let's submit the resource with:

bash

```
$ kubectl apply -f backend-v2.yaml
deployment.apps/backend-v2 created
```

If you inspect the logs from the front-end pod, you should

see responses from both backends:

```bash
$ kubectl logs frontend-6c64d7446-mmgpv
    <h1>v2.0.0</h1>
    <h1>v2.0.0</h1>
    <h1>v1.0.0</h1>
    <h1>v2.0.0</h1>
    <h1>v2.0.0</h1>
    <h1>v1.0.0</h1>
    <h1>v2.0.0</h1>
    <h1>v2.0.0</h1>
```

What about the dependency graph?

The Service Mesh was able to map both backends correctly.

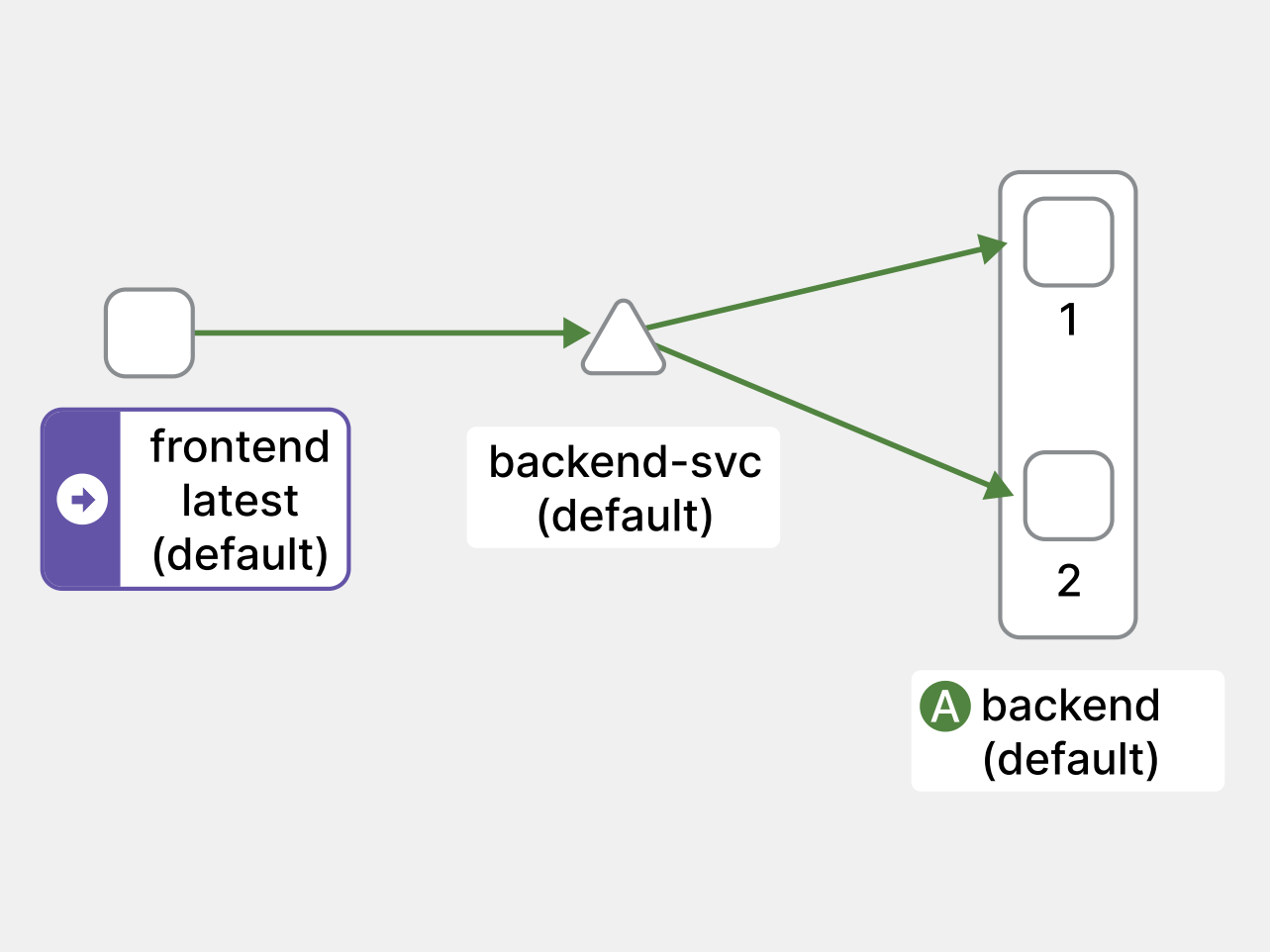


Fig. The network graph in Istio

This is already an excellent excuse to use a Service Mesh if you wish to map all connections between your apps.

But you can do so much more.

For example, what if the second version of the backend should only receive a fraction of the traffic?

This is a common requirement if you want to test a newer version of the service in production, but only with a smaller

pool of users.

Canary release

Let's say you want only 10% of the traffic to reach the backend-v2.

You will have to instruct the proxy on the frontend pod to only send one request every 10 to that backend.

The rest should still be routed to the backend-v1.

Unfortunately, issuing those instructions to the individual proxy is not always doable.

What if you have dozens of replicas of your front-end? Should you manually update all of the proxies?

For this reason, Service Meshes have two parts:

- **Some proxies are interested in routing the traffic.** This is usually referred to as the **data plane.**
- **A central API server collects and distributes your instructions to the proxies.** This is usually referred to as the **control plane.**

Istio's control plane can be controlled with Kubernetes Custom Resource Definitions (CRDs).

Here's an example of how you can split the traffic with an

`HTTPRoute`:

httproute.yaml

```
apiVersion: gateway.networking.k8s.io/v1beta1
kind: HTTPRoute
metadata:
  name: split
spec:
  parentRefs:
  - kind: Service
    name: backend-svc
    port: 80
  rules:
  - backendRefs:
    - name: backend-v1
      port: 80
      weight: 90
    - name: backend-v2
      port: 80
      weight: 10
```

Notice how there are three Services defined in the CRD (and you have created only one so far):

- `backend-svc` is the service that targets all Pods.
- `backend-v1` is the service that selects Pods from the backend-v1 deployment.
- `backend-v2` is the service that selects Pods from the backend-v2 deployment.

Istio uses Kubernetes Services not to route the traffic but as a mechanism to retrieve a list of Pod's IP addresses and ports.

Then, it combines the instructions from the CRD and

informs the proxies to route the traffic directly to the Pods.

Let's see it in action.

Let's create the missing services to target the individual versions:

services.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: backend-v1
  labels:
    app: backend
    version: "1"
spec:
  ports:
    - port: 80
      targetPort: 8080
  selector:
    app: backend
    version: "1"
---
apiVersion: v1
kind: Service
metadata:
  name: backend-v2
  labels:
    app: backend
    version: "2"
spec:
  ports:
    - port: 80
      targetPort: 8080
  selector:
    app: backend
    version: "2"
```

You can submit those to the cluster with:

```bash
$ kubectl apply -f services.yaml
service/backend-v1 created
service/backend-v2 created
```

Before submitting the `HTTPRoute` definition, open a new terminal session and observe the logs from the frontend Pod:

```bash
$ kubectl logs -f frontend-6c64d7446-mmgpv
    <h1>v2.0.0</h1>
    <h1>v1.0.0</h1>
    <h1>v2.0.0</h1>
    <h1>v2.0.0</h1>
    <h1>v1.0.0</h1>
```

Now submit the `HTTPRoute` file with:

```bash
$ kubectl apply -f httproute.yaml
httproute.gateway.networking.k8s.io/split created
```

Observe the output from the frontend Pod:

```bash
$ kubectl logs -f frontend-6c64d7446-mmgpv
    <h1>v1.0.0</h1>
    <h1>v1.0.0</h1>
    <h1>v1.0.0</h1>
```

```
    <h1>v1.0.0</h1>
    <h1>v2.0.0</h1>
    <h1>v1.0.0</h1>
    <h1>v1.0.0</h1>
    <h1>v1.0.0</h1>
    <h1>v1.0.0</h1>
```

The majority of the replies come from the backend v1.

Now let's change that.

What if we flip the traffic to 80% for the newer backend and leave only 20% for the old version?

Amend the weighting as follows:

httproute.yaml

```yaml
apiVersion: gateway.networking.k8s.io/v1beta1
kind: HTTPRoute
metadata:
  name: split
spec:
  parentRefs:
  - kind: Service
    name: backend-svc
    port: 80
  rules:
  - backendRefs:
    - name: backend-v1
      port: 80
      weight: 20
    - name: backend-v2
      port: 80
      weight: 80
```

And resubmit the definition to the cluster with:

bash

```
$ kubectl apply -f httproute.yaml
httproute.gateway.networking.k8s.io/split configured
```

Observe the output from the frontend Pod:

bash

```
$ kubectl logs -f frontend-6c64d7446-mmgpv
    <h1>v2.0.0</h1>
    <h1>v2.0.0</h1>
    <h1>v2.0.0</h1>
    <h1>v1.0.0</h1>
    <h1>v2.0.0</h1>
    <h1>v2.0.0</h1>
    <h1>v2.0.0</h1>
    <h1>v1.0.0</h1>
    <h1>v1.0.0</h1>
```

The majority of the responses have shifted to the backend v2 as expected!

As you can imagine, you can fully transition the traffic to reach backend-v2 with the following configuration:

httproute.yaml

```yaml
apiVersion: gateway.networking.k8s.io/v1beta1
kind: HTTPRoute
metadata:
  name: split
spec:
  parentRefs:
  - kind: Service
    name: backend-svc
    port: 80
  rules:
  - backendRefs:
```

```
    - name: backend-v1
      port: 80
      weight: 0
    - name: backend-v2
      port: 80
      weight: 1
```

You can apply the changes with:

bash

```
$ kubectl apply -f split.yaml
httproute.gateway.networking.k8s.io/split configured
```

And finally, observe the output with:

bash

```
$ kubectl logs -f frontend-6c64d7446-mmgpv
    <h1>v2.0.0</h1>
    <h1>v2.0.0</h1>
    <h1>v2.0.0</h1>
    <h1>v2.0.0</h1>
    <h1>v2.0.0</h1>
    <h1>v2.0.0</h1>
```

Excellent!

You gradually roll out traffic to the second backend.

Summary

In this article, you learned:

- How to deploy Istio.
- How to trace dependencies between microservices using Kiali.
- How to fine-tune the traffic reaching your apps using the `HTTPRoute`.